



정처기 감자 실기 코딩 압축 요약 PDF

0. 요약 범위

2026년 1회 예상 실기 코딩 문제 범위 요약

출제확률 : 100 : 99.9% ☆ : 80% 🔥 : 50% 😊 : 20% 🍌 : 필수

과목	예상 문제 수	예상 문제 범위
해기초	해기초	변수 자료형 출력함수 연산자
C언어	3~4문제	생기초 합수 변수스코프 조건문 반복문 배열 문자열 포인터 구조체 기초 typedef 입력함수 재귀함수 문자열포인터 구조체포인터 static 변수 #define string.h ctype.h 심화 이중포인터 2차원배열 malloc 연결리스트 큐 스택
자바	3~4문제	생기초 클래스와객체 상속 배열 문자열 접근제어자 기본제공클래스 참조형vs기본형변수 인터페이스 기초 다형성과부모자식대결 static 예외처리 for-each enum 심화 람다표현식 제네릭 추상클래스
파이썬	1~2문제	생기초 리스트 문자열 입력함수 딕셔너리 집합 튜플 기본제공함수 조건문 반복문 기초 컴프리헨션 언패킹 함수 클래스
총	11 ~ 12문제	

0. 해기초

각 언어에 해당하는 부분은 해당 언어 배치가 달림(공통이면 공통 배지)

1) 변수

해기초

변수 = 빈상자(아무거나 넣을 수 없는, 이름, 주소가 있는 빈상자)

C언어 **자바** **파이썬** *코드 → 코드를 그림으로 표현
- 모든 코드는 그림으로 표현 가능해야함
- 원본 코드가 코딩과 그림이랑 다를 수 있음

변수 선언 **변수 초기화** **변수 선언 + 초기화**

자료형 변수 이름

아무거나 넣지 못하게 함
→ 자료형과 동일한 자료형만
상자에 넣을 수 있음 주소 : 내맘대로 정할거임

초기화 : 변수에 처음으로 값을 넣는 것
초기화 = initialization(reset 아님)

파이썬 * 주소 사용안함

2) 자료형

해기초

C언어 자바 자료형 = 데이터 타입(data type)

자료형	설명	분류	추가설명	예시
int !	정수 = 소수점이 없는 수	정수	integer의 약자, 십진수	1
char !	문자 1개	정수(?)	아스키 코드 = 정수, character의 약자 'A'	
float !	실수 = 소수점이 있는 수	실수	floating point의 약자	1.1
long	큰 정수	정수	C언어 long보다 자바 long이 더 김	1000000000
double !	정밀한 소수	실수	double precision의 약자	1.1111111111
String !	문자열(문자 여러개)	문자열	참조형(자바 참조형vs기본형변수 참고) "Gamja"	
boolean !	참(true), 거짓(false)	boolean	참조형(자바 참조형vs기본형변수 참고) true	

파이썬 변수에 자료형을 따로 붙이지 않음. 외울 필요X.
하지만 C/자바와 비슷하게 변수의 자료형이 나뉘는 정도로 알아. (파이썬 - type 함수 참고)

3) 출력함수

해기초

C언어 출력함수 자바 출력함수 파이썬 출력함수

출력함수	설명
printf !	C언어 자바 "문자열", ["서식지정자"+변수 or 값], 자체 줄바꿈 없음!
System.out.print !	only 자바 왼쪽부터 + 로 결합, 문자열 만나면 자체 줄바꿈 없음!
System.out.println !	only 자바 문자열 결합, 뒤든 출력 자체 줄바꿈 있음!
putchar !	only C언어 문자 하나 출력
puts	only C언어 문자열 출력 printf("문자열")과 동일
print !	only 파이썬 뒤든 출력, 자체 줄바꿈 있음!, f-string(printf 서식지정자 비슷)
only 자바	*자바 출력 함수는 항상 System.out. 함수 형태로 사용(자바 클래스, 메서드 참고)

C언어 **자바** **파이썬** **printf(!)** **printf("문자열")**

*이전 모니터임
- 출력은 모니터에 보여줄거임

printf(문자열 아님)

int gamja = 1;

only C언어 printf("hello");
only 자바 System.out.printf("hello");

only C언어 printf(gamja);
only 자바 System.out.printf(gamja);

C언어 자바 서식지정자 !

서식지정자	왜 저렇게 표시하나	설명	예시 출력
%d !	decimal	정수	255
%c !	character	문자(1개)	A
%s !	string	문자열(문자 여러개)	Hello (** %s로 출력하려면 주소가 필요! 문자열 참고)
%f, %.1f, %.2f !	floating point	실수	255.000000, 255.0, 255.00
%x	hexadecimal	16진수 정수	ff(255의 16진수)
%o	octal	8진수 정수	377(255의 8진수)

C언어 자바 printf + 서식지정자 ! printf("서식지정자", 변수 or 값)

int gamja = 1;

only C언어 printf("%d", gamja);
only 자바 System.out.printf("%d", gamja);

gamja int 1

1

printf("서식지정자", 변수1, 변수2)

int gamja = 1;
int koguma = 2;

only C언어 printf("%d 그리고 %d", gamja, koguma);
only 자바 System.out.printf("%d 그리고 %d", gamja, koguma);

gamja int 1
koguma int 2

1 그리고 2

100 200

C언어 printf는 자체 줄바꿈이 없다 !

printf 두 번 호출

int gamja = 1;
int koguma = 2;

printf("%d", gamja);
printf("%d", koguma);

gamja int 1
koguma int 2

100 200

12

printf + 서식지정자 + \n

int gamja = 1;
int koguma = 2;

printf("%d\n", gamja);
printf("%d", koguma);

gamja int 1
koguma int 2

100 200

1
2

자바 print vs println printf와 달리 둘 다 뒤든 출력(int, float, 문자열 등등)

System.out.print(줄바꿈 없음)

int gamja = 1;
int koguma = 2;

System.out.print(gamja);
System.out.print(koguma);

gamja int 1
koguma int 2

100 200

12

System.out.println(print + \n)

int gamja = 1;
int koguma = 2;

System.out.println(gamja);
System.out.println(koguma);

gamja int 1
koguma int 2

100 200

1
2

자바 '+' 출력 결합 !

왼쪽부터 계산!
그런데 문자열 만나면 문자열 결합으로만 쪽 감
왼쪽이 정수니까 정수 + 정수 계산

int gamja = 1;
int koguma = 2;

System.out.println(gamja + koguma);
System.out.println("문자열" + gamja + koguma);

gamja int 1
koguma int 2

100 200

3
문자열12

처음부터 문자열이니까 문자열 + 문자열 계산== 이어 붙이기

C언어 putchar !

- 문자(char) 하나 출력

putchar('A');

문자는 홀따옴표!
출력될때 홀따옴표 없음

A

C언어 puts
- 문자열(string) 출력

puts("AB");

문자열은 쌍따옴표!
출력될때 쌍따옴표 없음

AB

파이썬 print ! 자바 System.out.print 비슷하게 뒤든 출력

*파이썬은 문자열/문자 구분이 없음, 전부 같은 str 자료형, 쌍따옴표/홀따옴표 구분 없음!
단, 쌍따옴표/홀따옴표 둘 다 한 방에 쓰는 경우 유의(파이썬 - 문자열 참고)

자체 줄바꿈 있음!

gamja = 1

print(gamja)
print(2)
print("감자")
print("있다")

1
2
감자
있다

심표로 여러개 출력

print(gamja, 2, "감자", "있다")
print의 기본 구분자는 공백(" ")

↑ 구분자: print에 들어간 값을 나눠주는(구분해주는)버석들

*공백이 사이 사이에 추가!

sep로 구분자를 바꿈

print(gamja, 2, "감자", "있다", sep=",")
print(gamja, 2, "감자", "있다", sep=" ")
print(gamja, 2, "감자", "있다", sep="")

1,2,감자,있다
12감자있다

end로 줄바꿈을 바꿈(?)

```
gamja = 1
print(gamja)
print(2)
print("감자")
print('있다')
```

→

```
print(gamja,end="\n")
print(2,end="\n")
print("감자",end="\n")
print('있다',end="\n")
```

12감자있다

```
print(gamja,end="")
print(2,end="")
print("감자",end="")
print('있다',end="")
```

1A2A감자A있다A

파이썬 f-string과 %포매팅

printf 서식지정자와 비슷하게 원하는 곳에 값을 넣어서 출력 가능

종류	모양	설명
f-string	<code>print(f"{{원든}}")</code>	중괄호 자리에 원든의 반환값이 들어감
%포매팅	<code>print("%지정자" % 들어갈거)</code>	서식지정자와 매우 비슷(옛날 거라)

파이썬 f-string

```
gamja = 1
koguma = 2
print(f"{{gamja}} 그리고 {{koguma}}")
```

순서는 중요지 않음(이름만 같으면 됨)

1 그리고 2

* 전부 같은 출력결과

C언어 서식지정자

```
int gamja = 1;
int koguma = 2;
printf("%d 그리고 %d", gamja, koguma);
```

순서가 중요함

파이썬 % 포매팅

```
gamja = 1
koguma = 2
print("%d 그리고 %d" % (gamja, koguma))
```

여러개 일때 tuple 사용

순서가 중요함

%지정자 (자리지정자) 받는 값

%s	string	문자열
%d	digit	정수
%f	float	실수

```
gamja = "감자"
print("%s 맛있다" % gamja)
```

문자열(string)이니까 %s

감자 맛있다

4) 연산자

산술 연산자 의미

추가설명	예시
+ , - ! 더하기, 빼기	1 + 2 = 3, 3 - 1 = 2
* ! 곱하기	역참조 연산자와 같은 기호 2 * 3 = 6
/ ! 나누기	C언어 자바 자료형에 따라 값이 달라짐 5 / 2 = 2 (정수끼리 나누면 결과값도 정수만 남음) 5 / 2.0 = 2.5 (실수가 들어가면 결과값은 실수)
only 파이썬 나눈 값은 항상 실수	5 / 2 = 2.5
// ! 나누기 only 파이썬 나눈 값은 항상 정수	5 // 2 = 2
% ! 나머지	나누고 몫을 제외한 나머지 5 % 2 = 1 (5 = 2 * 2 + 1)

대입 연산자 의미

= ! 오른쪽 값을 왼쪽 변수에 저장 (변수 = 값, a = 10) 수식 '!' 과 다름

대입 연산자

```
int gamja = 1;
gamja = gamja + 2;
printf("%d", gamja);
```

1 → 3

대입·복합 대입 연산자

```
int koguma = 0;
koguma = 0;
```

0 → 0

복합대입연산자 다르게 표현하면?

!	+= -=	a += 1은 a = a + 1, a -= 1은 a = a - 1
!	*= /= //=	a *= 2는 a = a * 2, a /= 2는 a = a / 2, a //= 2는 a = a // 2
	%=	a %= 3은 a = a % 3
	>>= <<=	a >>= 4는 a = a >> 4, a <<= 4는 a = a << 4

비교 연산자 의미

추가설명	예시
== ! 같다	1 == 1 (결과: 참), 1 == 2 (결과: 거짓)
!= ! 같지 않다 !는 보통 Not 의미	1 != 2 (결과: 참), 2 != 2 (결과: 거짓)
>, < ! 크다, 작다 >= (크거나 같다), <= (작거나 같다)	5 > 2 (결과:참) 5 < 2 (결과: 거짓) 1 < 2 (결과:참) 1 > 2 (결과: 거짓)

논리 연산자 조건

예시
&& ! and 두 조건 모두 참이면 참
! or 하나라도 참이면 참
! ! not 참은 거짓으로, 거짓은 참으로

* C언어 참 == 1, 거짓 == 0, 자바 참 == true, 거짓 == false, 파이썬 참 == True, 거짓 == False

비트 연산자

C언어 자바 파이썬 이진수(비트)로 변환해서 연산!

비트 연산자 의미

예시
& (AND) 두 이진수가 모두 1일때 1
(OR) 두 이진수 중 하나라도 1이면 1
^ (XOR) ! 두 이진수가 다를때 1
~ (NOT) 이진수를 반전(0은 1로, 1은 0으로)

자바 true, false엔 비트연산자 사용가능!

```
0 == 0 && 1 != 0 같은 true && true 결과 true
0 == 0 & 1 != 0 같은 true & true 결과 true
```

비트 연산자지만 사용 가능

시프트 연산자

C언어 자바 파이썬

시프트연산자 의미

예시
>> ! 이진수를 오른쪽>>으로 이동시킨다
<< 이진수를 왼쪽<<으로 이동시킨다

이진수

5를 이진수로 만들기 이진수를 오른쪽으로 이동

```
int right = 5 >> 1
int right = 이진수 101 >> 1
int right = 이진수 101 >> 1 == 이진수 10
```

5의 이진수 == 101

이진수를 십진수로 만들기

```
int right = 5 >> 1;
right가 int니까!
이진수 10 → 1*2^1 + 0*2^0 == 2
```

2^1 2^0

```
int left = 5 << 1;
left가 int니까!
이진수 1010 → 1*2^3+0*2^2+1*2^1+0*2^0 == 8+0+2+0 == 10
```

2^3 2^2 2^1 2^0

삼항연산자는 C언어 조건문, C언어 증감연산자는 반복문에서 확인!

연산자 우선순위 !

왼쪽이 제일 높음, 같은 열이라면 위가 더 높음 괄호>()는 수학, 코드 어디서든 항상 최상위!

비교 연산자	비트 연산자	논리 연산자	삼항 연산자	대입 연산자
< <= > >=	! &&	?:	+=	=
== !=	& ^	not	++	
== !=	~	and or	--	

* 파이썬 증감 연산자(++ , --) 없음!

1. C언어

1) 함수

용어

의미	예시
반환값	함수의 결과로 나온 값, return 값
반환형	return 값의 자료형
매개변수	함수가 외부에서 값을 받기 위한 변수
인자	함수로 넘기는 값

함수 예시

```
return 값의 자료형 함수이름 매개변수의 자료형 매개변수
반환형
int baht(int koguma){
    int can_koguma = koguma * 2;
    return can_koguma;
}
```

koguma(고구마)를 넣으면 2배로 캔 고구마(can_koguma)를 내놓는(return) 받(baht) 함수

can_koguma = koguma * 2;

can_gamja = baht(gamja);

printf("%d", can_gamja);

변수 정의 및 초기화

baht 함수

```
gamja int 1 100
koguma int 200
can_koguma int 300
can_gamja int 400
```

매개변수 대입

baht 함수

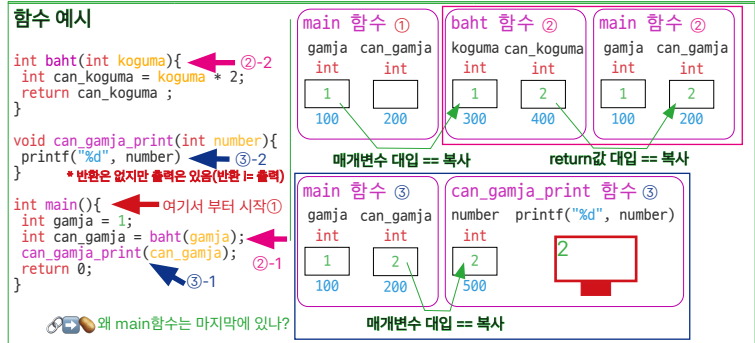
```
gamja int 1 100
koguma int 1 200
can_koguma int 2 300
can_gamja int 2 400
```

복사

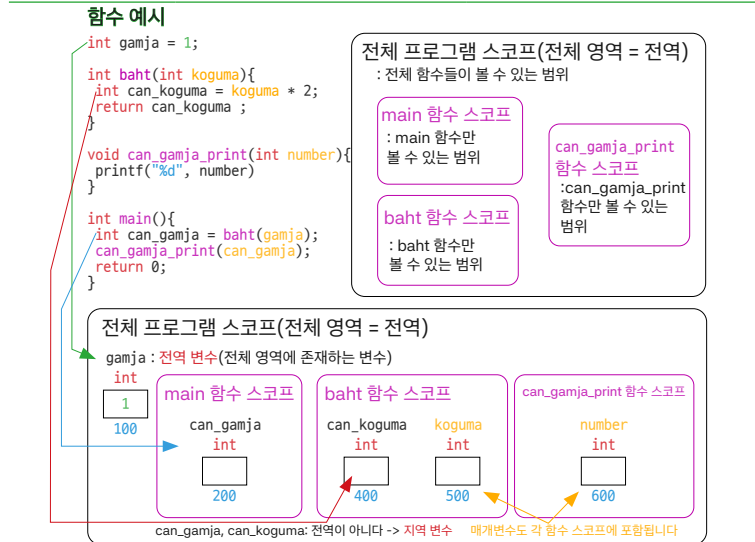
printf("%d", can_gamja);

2

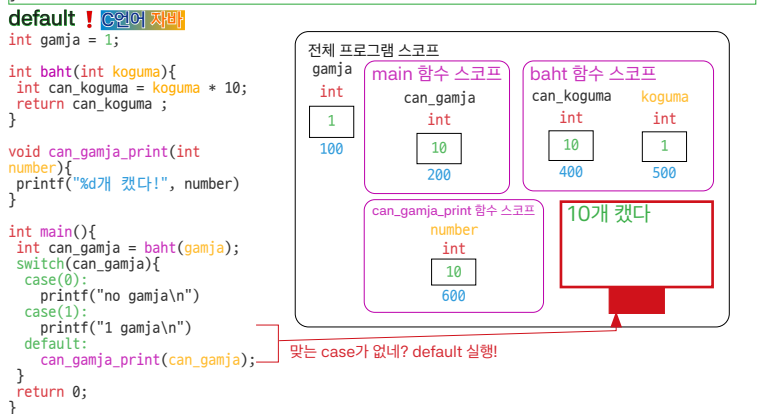
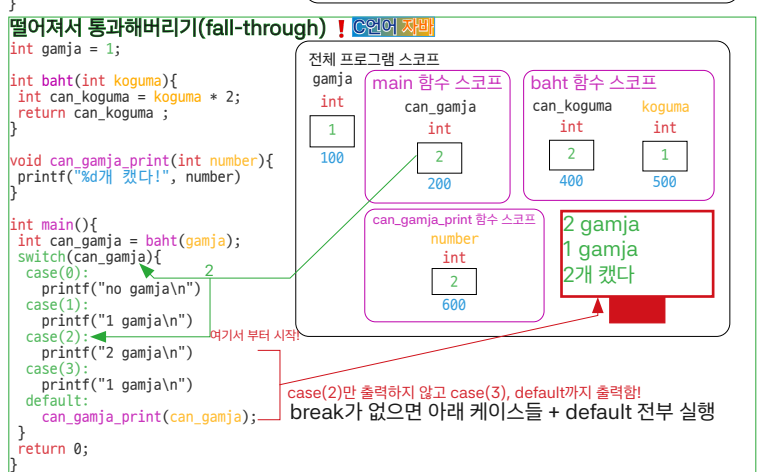
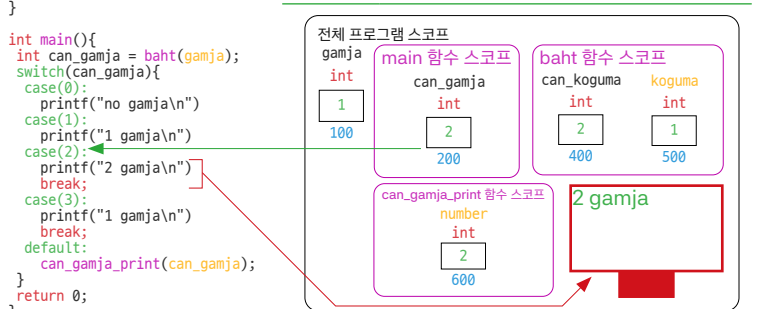
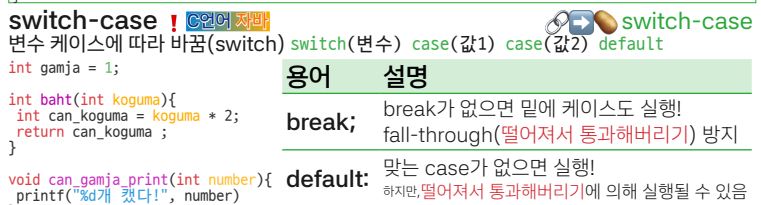
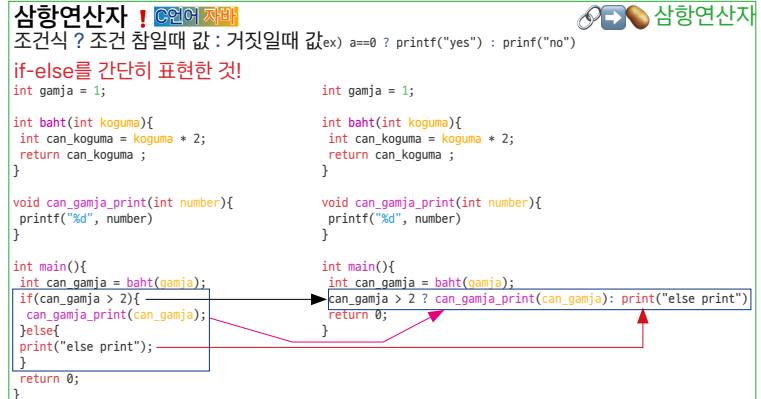
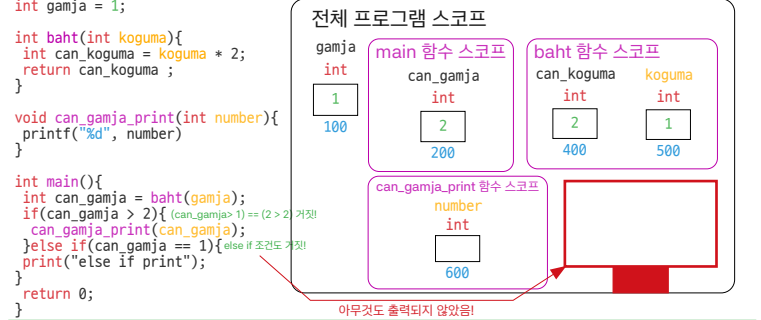
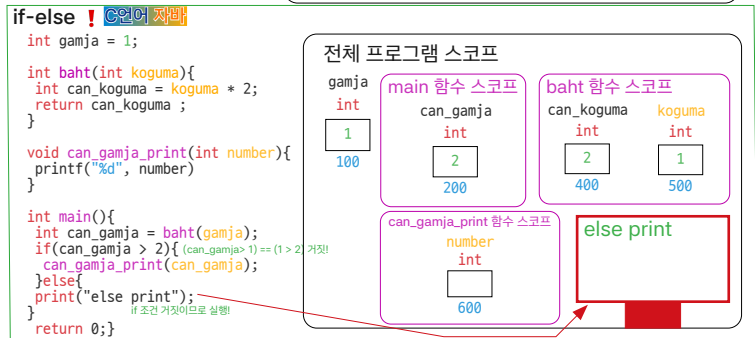
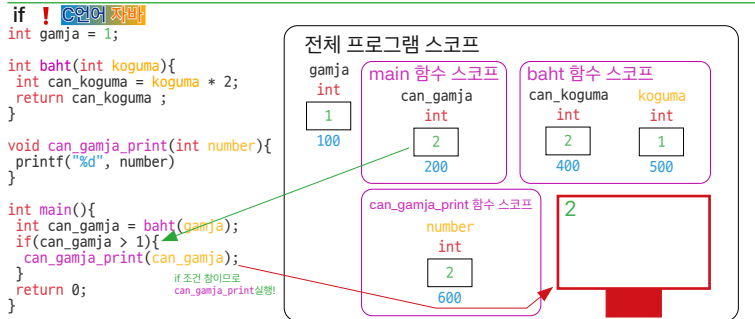
함수	설명	예시
main 함수 !	c언어 프로그램의 시작점	<code>int main(){ ... }</code>
사용자 정의 함수	개발자가 만든 함수(자유로운 함수이름)	<code>int baht(){ ... }</code>
void 함수 !	반환값(return)이 없는 함수 void = 텅 빈, 텅 빈 return 값을 갖는 함수	<code>void function(){ ... }</code>



scope	설명	세부 설명
telescope	망원경	tele(멀리) + scope(보다)
microscope	현미경	micro(작은 것) + scope(보다)
scope	보다(시야) 라는 의미를 갖고 있음	
변수 scope	함수들이 변수를 볼 수 있는 범위	== 함수가 사용 가능한 변수 범위 !



조건문	설명
if	조건이 참(1)이면 실행
else	if 조건이 거짓(0)이면 실행
else if	if 조건이 거짓이고 else if 조건에 맞는 경우 실행



4) 반복문 **생기초** **C언어 자바** **반복문**

반복문	설명	예시
for !	괄호에 초기식, 조건식, 증감식 모두 존재	for(i=0; i<2 ; i++)
while !	괄호에 조건식만 존재, 나머지는 내부에 존재	while(i<2)
break !	루프 즉시 종료	while(i<2){ if(i==1){break;}}
continue !	다음 반복으로 건너뛰기	while(i<2){ if(i==0){continue;}}

증감 연산자 **C언어 자바** **증감 연산자**

증감연산자	대입연산자(=) 혹은 [인덱스]에 있을때... 단독이면?
먼저+ 이따+ ++n n++	n에 1을 더함. 그런데, 위치에 따라 먼저 더하거나 이따 더함
먼저- 이따- --n n--	n에 1을 뺌. 그런데, 위치에 따라 먼저 빼거나 이따 뺌
	위치 의미없다!

대입연산자와 함께라면 !

```
int gamja = 1;
int gamjachuga = ++gamja
```

먼저+ 한 걸 대입!

단독 사용

```
int gamja = 1;
++gamja
```

gamja

인덱스

```
n = 1; arr[2]가 아니라 arr[1]에 접근
인덱스 직후 n이 1증가
```

for (i = 0; i < 3; ++i)

for (i = 0; i < 3; i++)

for문에서 단독 사용시 먼저, 이따 차이 없음!

5) 배열 **100** **생기초** **배열**

배열(array)==연속된 빈 상자 (변수==빈 상자)
배열 선언 *배열 선언에서 [] 와 인덱스 []는 다른 의미

```
int gamja_array[3];
```

배열 내 모든 빈 상자에는 int만 담을 수 있음!
배열의 크기 == 배열 상자 갯수(3개)
주소는 연속으로 해!
100, 101, 102(O)
100, 200, 300(X)
인덱스(Index)

배열 초기화

```
int gamja_array[3]={10,11,12};
```

전체 프로그램 스코프

배열이 있는 구조체

```
int gamja_array[]={10, 11, 12};
```

초기화하면서 배열 크기 생략 가능!
int gamja_array[]; 에러!
선언만 하면 배열 상자갯수 자동 지움!

배열이 있는 구조체 초기화

```
int gamja_array[3]={10};
```

int gamja_array[3]={10,0,0};

6) 포인터 **100** **기초** **포인터**

변수인데(빈 상자인데) 주소를 담아주는 변수(빈 상자)
포인터 선언
*은 포인터라는 의미

```
int *p;
```

다른 변수와 다르게 주소를 담아주니까 주소 역할(파라미터)으로 표시할거임

포인터는 무조건 주소를 담아줌!

포인터 주소는 1000단위로 할거임 (변수는 100단위, 이따와 문제풀때 안변경함)

내가 담아온 주소에 있는 것이 int다 (변수랑 다른 부분, 포인터는 항상 주소를 담아줌)

포인터 초기화

```
int gamja = 1;
int *p = &gamja;
```

&은 포인터에서 주소를 의미
&gamja==주소 100
&p==1000

배열을 가리키는 포인터 !

```
int gamja_array[3]={10,11,12};
int *p = gamja_array;
```

포인터에 배열 이름을 담으면 배열의 첫번째 상자 주소를 담게 됨!
(p == 주소100)

배열 이름은 주소다

포인터로 값 가져오기 !

```
int gamja = 1;
int *p = gamja;
printf("%d", *p);
```

*p와 int *p에서 *의 의미 다름!
*p == p 주소의 값
*p == p가 포인터선언

printf("%d", *(&gamja));

printf("%d", *(주소100));

주소 100의 값 == 1

포인터 산술 ! **포인터 산술**

***p+1은 뭐가?**

```
int gamja_array[3]={10,20,30};
int *p = &gamja_array;
printf("%d", *p+1);
printf("%d", *(주소100+1));
printf("%d", *p+10);
```

***(p+1)은 뭐가?**

```
printf("%d", *(p+1));
printf("%d", *(주소100+1));
printf("%d", *(주소101));
printf("%d", *p+20);
```

주소가 움직이느냐에 따라 달라지는 결과!

7) 구조체 **100** **기초** **구조체**

구조체	뭐 안에 뭐가 있고 이런 구조가 있음==구조체
struct	구조체 선언시 사용
멤버	구조체 내에 있는 변수, 배열 등
멤버 접근	멤버 값을 사용함(ex: modoom.gamja)

구조체 선언

```
struct Modoom{
int gamja;
int algamja;
int ssidamja;
};
```

전체 프로그램 스코프

구조체를 사용해봄

```
printf("%d", Modoom.gamja);
printf("%d\n", Modoom.algamja);
printf("%d", Modoom.ssidamja);
```

구조체 초기화
방법1 {중괄호} 사용 !

```
struct Modoom gamjas{1, 2, 3};
```


방법2 멤버 접근 사용 !

```
struct Modoom gamjas;
gamjas.gamja = 1;
gamjas.algamja = 2;
gamjas.ssidamja = 3;
```

전체 프로그램 스코프

구조체를 포인터로 사용해봄 !

```
struct Modoom{
int gamja;
int algamja;
int ssidamja;
};

struct Modoom gamjas{1, 2, 3};

struct Modoom *p = &gamjas;
```

내가 담아온 주소에 있는 것이 struct Modoom 이다 (내가 담아온 주소로 가면 Modoom 구조체 밖에 있음거림)

printf("%d", p->gamja);

==

printf("%d", 주소100->gamja);

-> 주소에 있는 구조체의 멤버에 접근(화살표 연산자) !

주소100->gamja : 주소100에 있는 구조체의 gamja 멤버에 접근

배열이 있는 구조체

배열이 있는 구조체 선언

```
struct Modoom{
int gamja_array[3];
int algamja;
int ssidamja;
};
```

전체 프로그램 스코프

배열이 있는 구조체 초기화

```
struct Modoom gamjas{{1,2,3}, 4, 5};
```

전체 프로그램 스코프

구조체 내에는 배열, 구조체, 포인터 뭐든 들어갈 수 있음! 배열이 들어간 것처럼 뭐든 넣어서 그릴 수 있어야함!

8) typedef **기초** **typedef**

자료형에 별명 붙이기
typedef 자료형 별명;

```
typedef int G;
G gamja = 10;
```

별명을 붙이는 이유는 보통 사용할때 짧게 사용하기 위함
(이는 원래 원래에서 이렇게 별명을 붙이지 않음
그래서 보통은 각 자료형에 사용됨, 보통 구조체)

구조체 typedef !

```
struct Modoom{
int gamja;
int algamja;
int ssidamja;
};

typedef struct{
int gamja;
int algamja;
int ssidamja;
}Modoom;
```

struct Modoom gamjas1; struct Modoom gamjas2;

Modoom gamjas1; Modoom gamjas2;

9) 문자열 **기초** **문자열**

문자열(string)==연속된 빈 상자(배열)에 문자들이 들어가 있음==배열!
문자열 선언

```
char gamja_name[]="GJ"
```

문자열은 쌍따옴표!
문자열 상자에는 문자(character == char) 들어감
그런데! 마지막에는 항상 '\0'(NULL)이 들어감
NULL == 비어있다
'\0'도 문자임(아스키 코드 = 0)

전체 프로그램 스코프

문자열 출력

```
printf("%s", gamja_name)
```

문자==아스키코드==정수 !

문자	아스키 코드	추가 설명
'/0'	0	
'A'	65	대문자 A는 65 기억해두기!
'B'	66	A 다음 B, 1번 다음이니가 65에서 +1
'a'	97	소문자 a는 97 기억해두기!
'b'	98	a 다음 b, 1번 다음이니가 97에서 +1

C는? A 다음 B 다음 C, 2번 다음이니가 65+2==67
c는? a 다음 b 다음 c, 2번 다음이니가 97+2==99

그래서! A, B의 아스키 코드만 알면 모든 문자 아스키코드 알 수 있음!

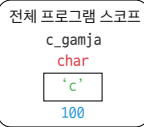
문자 아스키 코드 F70d100

'F' 70 A 65가 안외워지면 F 70을 외웁시다!
'd' 100 a 98이 안외워지면 d 100을 외웁시다!

C는? F 이전 E 이전 D 이전 C, 3번 이전이니까 70-3==67
c는? d 이전 c, 1번 이전이니까 100-1==99

문자 선언

char c, gamja='c';
문자는 홀따옴표!



문자 출력

printf("%c\n", c_gamja)



printf("%d", c_gamja)



문자열 접근

포인터로 접근!

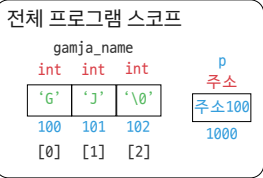
주소가 있으니 포인터로도 접근 가능!

char *p = gamja_name

printf("%s", *p)

printf("%c", *주소100)

printf("%c", 'G')



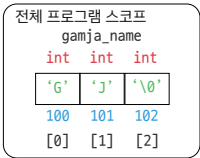
인덱스로 접근!

문자열==배열이라 배열처럼 인덱스로 접근 가능!

char gamja_name[]="GJ"

printf("%c\n", gamja_name[0])

printf("%c\n", gamja_name[1])



문자열 출력(포인터)

printf("%s", 주소)

%c에는 *p(주소)이 필요했지만 %s에는 주소가 필요!

printf("%s", p)

주소에 있는 모든 문자들을 출력해줌(NULL은 비어있으니 출력X)



10) string.h 기초

문자열 처리 함수 모음(라이브러리)

함수	의미	s 값	결과
strlen(s) !	s 문자열의 길이	"Gam"	3
strcpy(d, s)	s 문자열을 d에 복사	"Gamja"	"Gamja"
strcmp(s1, s2)	s1, s2 같으면 0	s1 값 "abc" s2 값 "abc"	0
strcat(s1, s2)	s1뒤에 s2 붙임	s1 값 "ab" s2 값 "cd"	"abcd"

11) 재귀함수 기초

다시(재) 돌아오다(귀), 나에게 다시 돌아오는 함수

int gamja = 2

```
int plus(int num){
    if (num <=0) return 0;
    return num + plus(num-1)
}
```

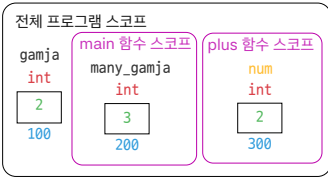
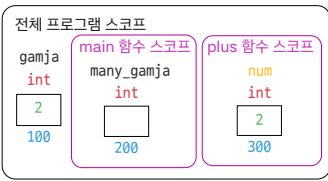
```
int main(){
    many_gamja=plus(gamja)
    printf("%d", many_gamja)
}
```

```
int plus(int 2){
    if (2 <=0) return 0;
    return 2 + plus(2-1)
}
```

```
int plus(int 1){
    if (1 <=0) return 0;
    return 1 + plus(1-1)
}
```

```
int plus(int 0){
    if (0 <=0) return 0;
    return num + plus(0-1)
}
```

마지막 return이 나올때까지 꼭 내려갔다 다시 돌아오면 됨!



12) 입력함수 기초

입력함수 설명 서식지정자는 출력함수 부분을 확인!

scanf ! "서식지정자"+주소, printf와 달리 주소가 필요!(단, 문자열은 주소X)

getchar 문자 하나 입력

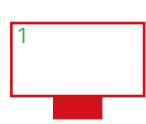
gets ! 문자열 입력

scanf(서식지정자, 주소)

int gamja;

scanf("%d", &gamja); !

printf("%d", gamja);



scanf("%s", 문자열);

char gamja[3];

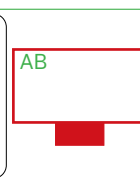
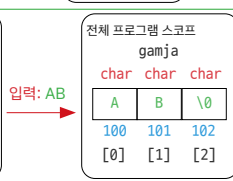
scanf("%s", gamja);

printf("%s", gamja);

배열 이름은 주소다

문자열==배열(문자가 들어가는)

그래서 문자열은 &gamja가 아닌 gamja가 들어감!

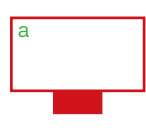
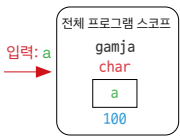


getchar()

char gamja;

gamja = getchar();

printf("%c", gamja);

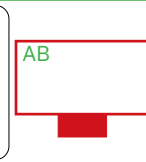
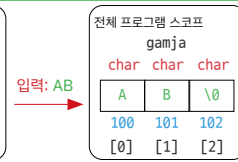
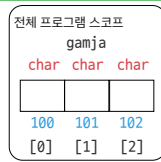


gets() !

char gamja[3];

gets(gamja);

printf("%s", gamja);



13) 문자열 포인터 기초

문자열 == 배열 잊지마!

복습! 배열을 가리키는 포인터

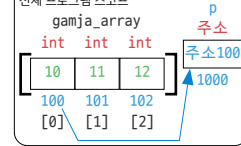
int gamja_array[3]={10,11,12};

int *p = gamja_array;

포인터에 배열 이름을 담으면 배열의 첫번째 상자 주소를 담게 됨!

(p == 주소100)

배열 이름은 주소다

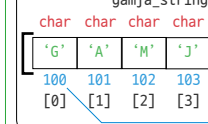


문자열을 가리키는 포인터 !

char gamja_string[] = "GAMJA";

char *p = gamja_string;

전체 프로그램 스코프



printf("%c", *p);

printf("%c", *주소100);

printf("%c", 'G');

printf("%c", *(p+3));

printf("%c", *(주소100+3));

printf("%c", *(주소103));

printf("%c", 'J');

printf("%s", p);

printf("%s", 주소100);

주소 100부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

주소 103부터 '\0'까지 출력

14) 구조체 포인터 기초

구조체를 가리키는 포인터 !

struct Modoom{

int gamja;

int algamja;

};

struct Modoom *p;

전체 프로그램 스코프

Modoom

gamja int

algamja int

주소 200 300 1000

100

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

구조체를 가리키는 구조체 멤버 포인터 !

struct Modoom{

int gamja;

struct Modoom *modoom_p;

};

내가 담아둔 주소에 있는 것이 struct Modoom이다!

Modoom이라는 구조체 내에는 Modoom 구조체를

가리킬 수 있는 포인터 modoom_p가 멤버로 있는 거임!

전체 프로그램 스코프

Modoom

gamja int

modoom_p 주소

주소 200 1000

100

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

화살표 연산자 -> !

포인터로 멤버 접근시 사용!

구조체에서 멤버 접근()과 구분하기 gamjas, gamja

struct Modoom gamjas = {1,2};

p = &gamjas;

printf("%d", p->gamja);

전체 프로그램 스코프

gamjas

gamja int

algamja int

주소 1 2 주소100 주소1000

100

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

주소 1000

16) 2차원 배열 🔥 심화 !

배열(array)==연속된 빈 상자 (변수==빈 상자)

2차원 배열 == 연속된 빈 상자인데 가로뿐만 아니라 세로로도 연속!

2차원 배열 선언

```
int gamja_2d[2][3];
[행][열]
[2][3]==행 2개, 열 3개
주소는
가로는 연속(100, 101, 102)
세로는 피아세(100, 200)
[행][열]
[2][3]==행 2개, 열 3개
주소는
가로는 연속(100, 101, 102)
세로는 피아세(100, 200)
```

전체 프로그램 스코프

```
gamja_2d
int int int
100 101 102
[0][0] [0][1] [0][2]
[1][0] [1][1] [1][2]
200 201 202
gamja_2d==gamja_2d[0]==&gamja_2d[0][0]==주소100
gamja_2d[1]==&gamja_2d[1][0]==주소200
int gamja_2d[2][3]={1,2,3,4,5,6};
같은 결과!
{1행} {2행}
```

2차원 배열 초기화

```
int gamja_2d[2][3]={1,2,3,4,5,6};
가로로 행부터 채워주기
print("%d", gamja_2d[0][2]);
같은 결과!
{1행} {2행}
```

전체 프로그램 스코프

```
gamja_2d
int int int
100 101 102
[0][0] [0][1] [0][2]
[1][0] [1][1] [1][2]
200 201 202
gamja_2d==gamja_2d[0]==&gamja_2d[0][0]==주소100
gamja_2d[1]==&gamja_2d[1][0]==주소200
int gamja_2d[2][3]={1,2,3,4,5,6};
같은 결과!
{1행} {2행}
```

포인터 배열로 2차원 배열 접근 !

```
포인터 배열 == 포인터를 담은 배열
int *pointer_array[2];
int gamja_2d[2][3]={1,2,3,4,5,6};
pointer_array[0]=gamja_2d[0];
pointer_array[1]=gamja_2d[1];
printf("%d", pointer_array[1][1]);
printf("%d", 주소200[1]);
printf("%d", 주소200+1);
printf("%d", 주소201);
```

전체 프로그램 스코프

```
pointer_array
주소 주소
주소100 주소200
1000 1001
[0] [1]
gamja_2d
int int int
100 101 102
[0][0] [0][1] [0][2]
[1][0] [1][1] [1][2]
200 201 202
같은 결과!
{1행} {2행}
```

17) 연결리스트 🔥 심화

연결되어있는 리스트(목록), 빈 상자들이 연결되어있는 형태!

```
struct Modoom{
int gamja;
struct Modoom *modoom_p;
};
struct Modoom gamjas1 = {1, NULL};
struct Modoom gamjas2 = {2, &gamjas1};
struct Modoom gamjas3 = {3, &gamjas2};
struct Modoom *p = &gamjas3;
```

빈상자가 붙어있는 게 배열이라면
배열보다 느슨하게 연결되어있는 형태를 연결리스트라고 함!
보통 포인터를 활용해서 연결!
포인터가 빈상자를 연결하는 실이라고 생각하자!

전체 프로그램 스코프

```
gamjas1 gamjas2 gamjas3 p
gamja modoom_p int 주소
1 NULL 200 1000
2 주소100 400 2000
3 주소300 600 3000
주소 주소
주소100 주소200 주소300 주소400
```

연결리스트를 돌아보기 !

반복문과 포인터를 사용하면 연결리스트의 값들을 주르륵 불러올 수 있음

```
while(p){
printf("%d", p->gamja);
p = p->modoom_p;
}
```

while 반복 1회차

p=주소500

3

전체 프로그램 스코프

```
gamjas1 gamjas2 gamjas3
gamja modoom_p int 주소
1 NULL 200 1000
2 주소100 400 2000
3 주소300 600 3000
주소 주소
주소100 주소200 주소300 주소400
while(p){
printf("%d", 주소500->gamja);
p = 주소500->modoom_p;
}
while(p){
printf("%d", 3);
p = 주소300;
}
대입하기 전임!

```

while 반복 2회차

p=주소300

32

전체 프로그램 스코프

```
gamjas1 gamjas2 gamjas3
gamja modoom_p int 주소
1 NULL 200 1000
2 주소100 400 2000
3 주소300 600 3000
주소 주소
주소100 주소200 주소300 주소400
while(p){
printf("%d", 주소300->gamja);
p = 주소300->modoom_p;
}
while(p){
printf("%d", 2);
p = 주소100;
}
대입하기 전임!

```

while 반복 3회차

p=주소100

321

전체 프로그램 스코프

```
gamjas1 gamjas2 gamjas3
gamja modoom_p int 주소
1 NULL 200 1000
2 주소100 400 2000
3 주소300 600 3000
주소 주소
주소100 주소200 주소300 주소400
while(p){
printf("%d", 주소100->gamja);
p = 주소100->modoom_p;
}
while(p){
printf("%d", 1);
p = NULL;
}
대입하기 전임!

```

while 반복 4회차

p=NULL==거짓

이므로 반복 끝!

18) 큐 🔥 심화

queue==줄, 대기열. 줄서면 앞사람이 먼저나가고 뒤로 사람이 들어옴

구조체 + 배열 + 포인터를 활용해서 큐 형태를 만들 수 있음!(한 번 이해하기만 하자)

```
int arr[] = {1, 2, 0, 0, 0};
```

① 큐를 다루는 Queue 구조체

```
typedef struct {
int* queue;
int front;
int rear;
int size;
} Queue;
Queue q = {arr, 0, 2, 5};
큐 == arr을 다루기 위한 포인터
arr의 길이
```

Queue 구조체가 큐를 어떻게 다루나 보자

② 앞사람 내보내기(진짜로 내보내진 않음)

dequeue(&q) 코드 순서는 이해를 돕기 위해
언어 순서에 맞지 않게 썼음. 그냥 순서대로 보심

```
int dequeue(Queue* q) {
int data = q->queue[(q->front)];
q->front++;
return data;
}
arr 배열에서 앞사람이 먼저 나가고
뒤로 사람이 들어올 수 있도록 할거임(그게 큐)
```

2번째 내보내기

```
dequeue(&q)
```

```
int dequeue(Queue* q) {
int data = q->queue[(q->front)];
q->front++;
return data;
}
arr[0]; 맨 앞(front)에 있는 사람을 사용함
arr[1]; 두번째에 있는 사람을 사용함
```

③ 뒤로 사람 들어오기

```
enqueue(&q, 3)
void enqueue(Queue* q, int data) {
q->queue[(q->rear)] = data;
q->rear++;
return data;
}
arr[2] = 3;
```

전체 프로그램 스코프

```
arr Queue q
int int int int int
1 2 0 0 0
[0] [1] [2] [3] [4]
queue int * front int rear int size
주소100 300 1 2 5
400 500 600
200
q->front++
arr[0]; 맨 앞(front)에 있는 사람을 사용함
arr[1]; 두번째에 있는 사람을 사용함
enqueue(&q, 4)
void enqueue(Queue* q, int data) {
q->queue[(q->rear)] = data;
q->rear++;
return data;
}
arr[3] = 4;
rear가 이동하면서 마치 뒤로 사람이 들어오는거 같은 효과가 나타남

```

코드를 제대로 쓰면 아래와 같음

```
int arr[] = {1, 2, 0, 0, 0};
```

```
typedef struct {
int* queue;
int front;
int rear;
int size;
} Queue;
```

```
Queue q = {arr, 0, 2, 5};
```

```
int dequeue(Queue* q) {
int data = q->queue[(q->front)];
q->front++;
return data;
}
```

```
int main(){
dequeue(&q);
dequeue(&q);
enqueue(&q, 3);
enqueue(&q, 4);
return 0;
}
```

! 시험팁!

Queue, enqueue, dequeue 등의 단어가 보이면
큐 형태겠구나 생각하고 문제를 풀어보자
(스택일수도 있음)

```
void enqueue(Queue* q, int data) {
q->queue[(q->rear)] = data;
q->rear++;
}
```

```
int main(){
dequeue(&q);
dequeue(&q);
enqueue(&q, 3);
enqueue(&q, 4);
return 0;
}
```

19) 스택 🔥 심화

프링글스 감자칩처럼 한쪽(뚜껑있는 곳)으로만 넣고 빼기가 가능!

배열만 활용해서 스택 형태를 만들 수 있음!(한 번 이해하기만 하자)

```
int arr[5] = {0};
int top = -1;
```

```
void push(int data) {
arr[++top] = data;
}
```

```
int pop() {
return arr[top--];
}
```

```
int main(){
push(10);
push(20);
printf("%d\n", pop());
printf("%d\n", pop());
return 0;
}
```

```
! 시험팁!
push, pop, into, take
등의 단어가 보이면
스택 형태겠구나 생각
하고 문제를 풀어보자
(enqueue, dequeue
도 스택일수도 있음)
```

```
return 0;
return arr[1];
return arr[0];
```

```
실제로 뺐건 아니지만
뚜껑있는쪽부터 순차적 반환
```

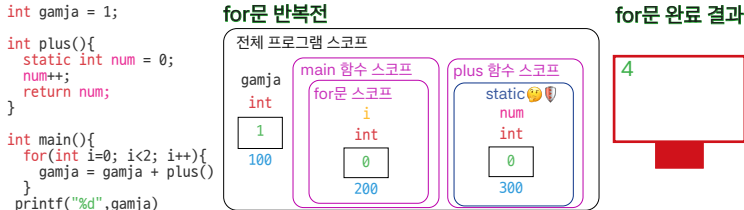
```
arr
int int int int int
10 20 0 0 0
[0] [1] [2] [3] [4]
top
int
-1
++top
0
1
0
-1
```

```
arr
int int int int int
10 20 0 0 0
[0] [1] [2] [3] [4]
top
int
-1
++top
0
1
0
-1
```

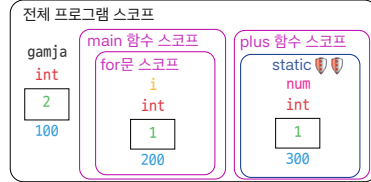
```
arr
int int int int int
10 20 0 0 0
[0] [1] [2] [3] [4]
top
int
-1
++top
0
1
0
-1
```

20) static 변수 🤔 심화

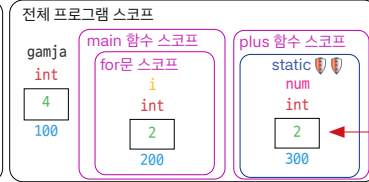
시작 시 한번만 생겨나고 계속 유지(고정)되는 외딴섬 변수
 계속 유지되기 때문에 반복할때 일반적인 변수들과 다른 모습을 보임!



for문 1회 반복후



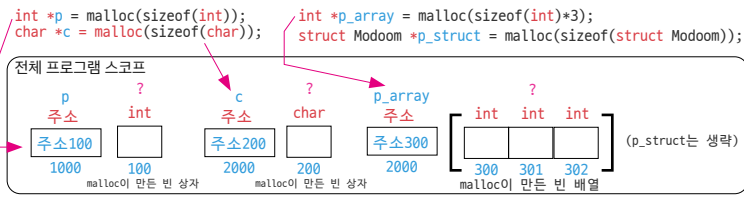
for문 2회 반복후



21) malloc() 🤔 심화

malloc은 기본적으로 주소를 반환
 (+ 괄호에는 메모리 사이즈가 들어가는데 중요하지 않음)
 그러므로, 주소를 저장하는 포인터에 대입

malloc이 반환하는 주소는 빈 상자가 있는 주소(빈 배열, 빈 구조체 주소일 수도 있음)

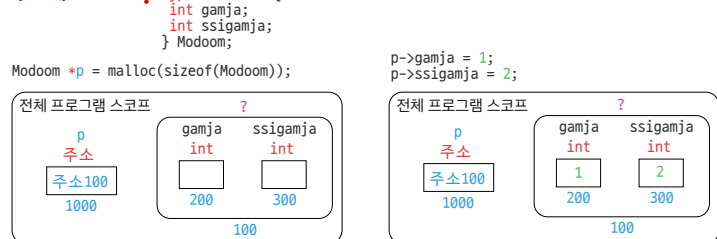


(형변환)malloc !

다른 자료형으로 사용하는걸 명시하기 위해 (자료)형변환 사용
 하지만, (자료)형변환은 생략해도 상관없음, 자동으로 포인터 자료형에 맞춰서 형변환 됨

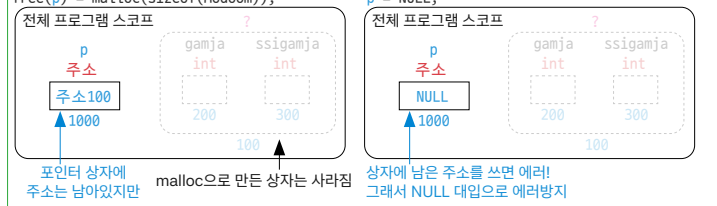
int *p = (int*)malloc(sizeof(int)); == malloc(sizeof(int));

구조체 malloc ! typedef struct{



free() !

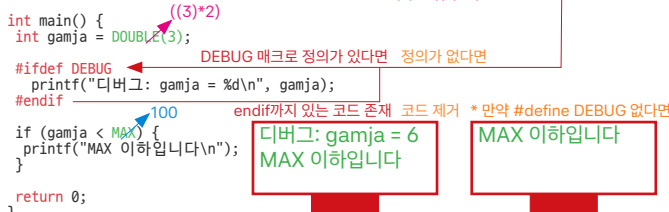
포인터가 저장해놓은 주소에 있는, malloc으로 만든 빈 상자가 사라집니다



22) #define 🤔 심화

#define으로 특정 이름을 정의(매크로 정의)하거나 바꿀 수 있음(매크로 상수, 매크로 함수).

#define DEBUG → 매크로 정의
 #define MAX 100 → 매크로 상수: MAX를 100으로 치환!
 #define DOUBLE(x) ((x) * 2) → 매크로 함수: DOUBLE(x)를 ((x)*2)로 치환



23) ctype.h 🤔 심화

문자 판별, 변환 함수가 들어있는 함수 모음(라이브러리) → 문자열 사용안됨

함수	의미	c1 값	결과	추가설명
isupper(c1) !	c1이 대문자인가?	'A'	참	
islower(c1) !	c1이 소문자인가?	'a'	참	
isdigit(c1) !	c1이 숫자인가?	'1'	참	isdigit(1) 아스키 코드 1은 숫자X. 거짓!
isalpha(c1)	c1이 알파벳인가?	'a'	참	대소문자 상관없이 알파벳이면 참
isspace(c1)	c1이 공백문자인가?	'\n'	참	공백문자 스페이스(' '), 탭('/t'), 줄바꿈('/n')
toupper(c1)	c1을 대문자로 바꿈	'a'	'A'	
tolower(c1)	c1을 소문자로 바꿈	'A'	'a'	

2. 자바

1) 클래스와 객체 🤔 심화

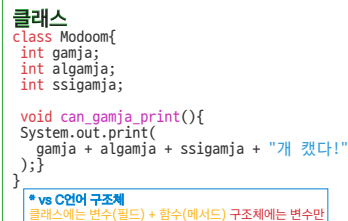
자바는 J라서 될 때 항상 설계도 필요함예) 배열 선언

설계도 == 클래스(Class)

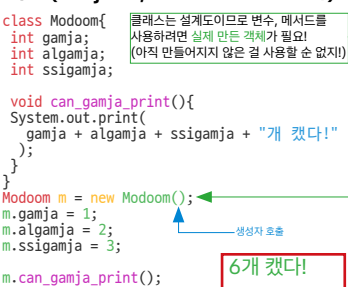
설계도로 실제 만든거 == 객체(Object)

클래스(Class, 설계도)

C언어의 구조체 선언과 비슷하지만 다름



객체(Object, 설계도로 만든 거)

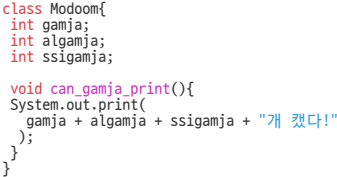


생성자

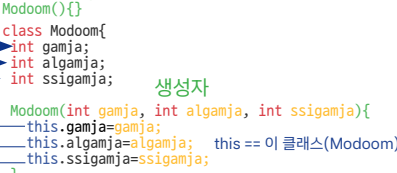
객체를 만들때 어떻게 생성할 것인지(어떻게 초기화할 것인지 정함)

Modoom m = new Modoom();

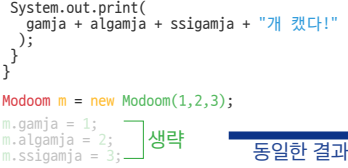
*사실 이 클래스에는 생성자가 생략되어 있음



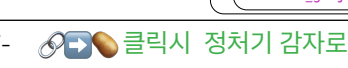
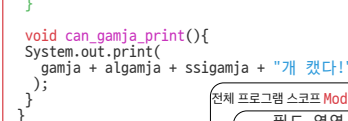
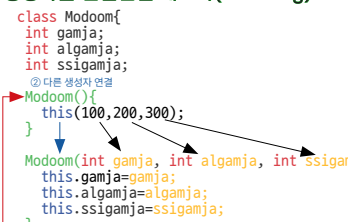
생성자를 넣어서 객체를 다르게 초기화해보자



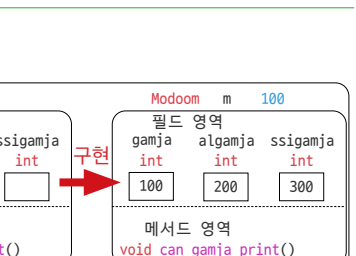
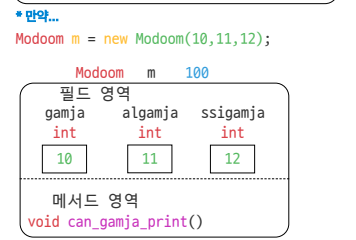
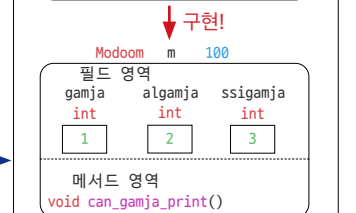
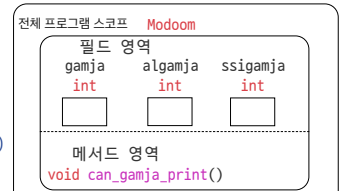
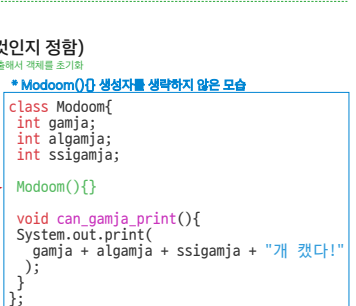
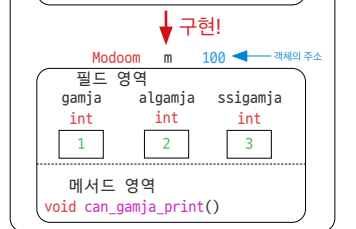
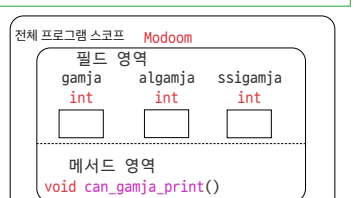
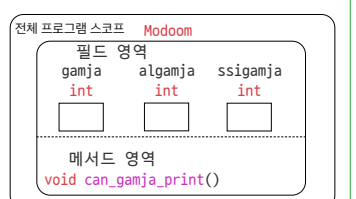
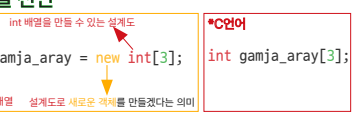
생성자를 연결연결 해보자(chaining)



생성자를 연결연결 해보자(chaining)



객체와 클래스



2) static 기초

static == "고정된", 클래스에 static (변수 혹은 메서드) == 클래스에 고정된 (변수 혹은 메서드)!
클래스는 설계도이므로 변수, 메서드를 사용하려면 실제 만든 객체가 필요! 했는데...

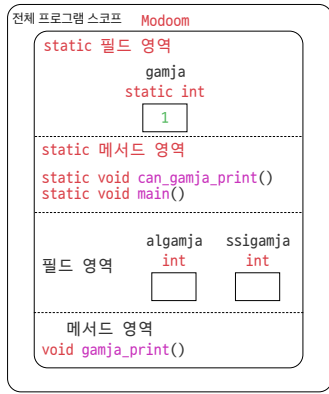
굳이 객체를 만들지 않아도 이미 클래스에 static 변수, 메서드가 고정되어 만들어져 있어서
직접 바로 사용가능함!

```
class Modoom{
    static int gamja = 1;
    int algamja;
    int ssigamja;

    static void can_gamja_print(){
        System.out.print(
            gamja + "개 됐다!"
        );
    }
    void gamja_print(){
        System.out.print(
            gamja + "개 있음"
        );
    }
    public static void main(String[] args){
        System.out.println(gamja);
        Modoom.can_gamja_print();
    }
}
```

1
1개 됐다!

main 메서드엔 반드시 static!
main 메서드가 속한 클래스의 객체를 만들지 않고 main 메서드를 사용하기 위해! 예시 코드를 보면 class Modoom 에 대한 객체가 없지만 main 메서드는 잘 돌아감



3) 상속

상속을 통해 부모 클래스의 설계도를 토대로 자식 클래스를 만들 수 있음

상속을 받으면서 또 자신만의 무언가로 넓혀갈 수 있음(extends): 메서드 추가, 필드 추가 등

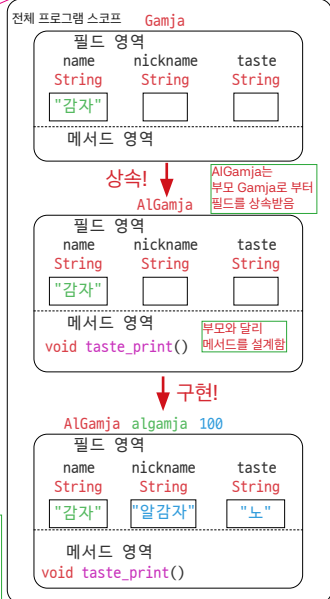
```
class Gamja{
    String name = "감자";
    String nickname;
    String taste;
}

class ALGamja extends Gamja{
    void taste_print(){
        System.out.print( taste + "맛");
    }
}

class Main{
    public static void main(String[] args){
        ALGamja algamja = new ALGamja();
        algamja.nickname="알감자";
        algamja.taste="노";

        System.out.println(algamja.name);
        System.out.println(algamja.nickname);
        algamja.taste_print();
    }
}
```

부모 클래스 Gamja
자식 클래스 ALGamja



super

super는 부모 클래스를 지칭

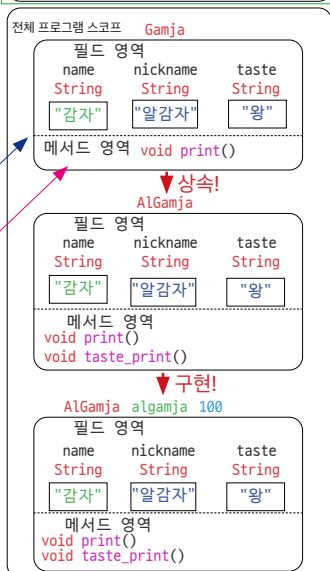
```
class Gamja{
    String name = "감자";
    String nickname;
    String taste;

    Gamja(String nickname, String taste){
        this.nickname = nickname;
        this.taste = taste;
    }
    void print(){
        System.out.println(name);
    }
}

class ALGamja extends Gamja{
    ALGamja(){
        super("알감자", "왕");
    }
    void taste_print(){
        super.print();
        System.out.print( taste + "맛");
    }
}

class Main{
    public static void main(String[] args){
        ALGamja algamja = new ALGamja();
        algamja.taste_print();
    }
}
```

감자
알감자
노맛



4) 인터페이스

클래스가 반드시 구현해야 할 메서드 목록 먼저 정의

클래스는 인터페이스 메서드를 구현함(implements)

```
interface Edible {
    void eat(String who);
}

class Gamja implements Edible {
    @Override
    public void eat(String who) {
        System.out.println(who + "가 감자를 먹음");
    }
}

class Main {
    public static void main(String[] args) {
        Gamja e1 = new Gamja();
        e1.eat("철수");
    }
}
```

인터페이스 대로
메서드 구현
(인터페이스 메서드는 기본적으로 public)

철수가 감자를 먹음

Note

5) 다형성과 부모 자식 대결

기초

오버라이딩과 static 메서드

다형성 == 여러가지 형태를 가질 수 있음. 뭐가? 객체와 메서드가...

① 부모인데 객체는 자식

Gamja g = new AlGamja();

타입은 부모(Gamja), 객체는 자식(AlGamja) -> 다형성!

문제는 이럴때 부모편과 자식편을 알아야 함

구분	어디편? 왜?
메서드	자식편 오버라이딩 (상속이면 부모편)

static 메서드 부모편 static은 클래스(설계도) 고정
→ 객체(실체) 안봄
→ 타입(클래스, 설계도)만 봄

필드 부모편 오버라이딩 개념 없음
static은 클래스(설계도) 고정
→ 객체(실체) 안봄
→ 타입(클래스, 설계도)만 봄

static 필드 부모편 static은 클래스(설계도) 고정
→ 객체(실체) 안봄
→ 타입(클래스, 설계도)만 봄

*메서드라고 다 부모편

② 오버라이딩 (over-riding)

자식이 부모의 말(메서드)을 뺏아타거임

class Gamja {
void print() { "나는" + name; }
}
class AlGamja extends Gamja {
void print() { "우리는" + name; }
}

→ 부모 자식 메서드가 같네!

class AlGamja extends Gamja {
void print() {
System.out.println("우리는" + name);
}
}

→ 자식 메서드를 사용

③ 오버로딩 (over-loading)

메서드 이름은 같은데 여러 방식으로 사용하게 함

매개변수를 추가하거나 (더 실거냐) 바꿈

메서드가 다양한 형태(다형성)를 가능하게 함

class Gamja {
void eat(String who) {
System.out.println(who + "가 먹음");
}
void eat(String who, int count) {
System.out.println(who + "가 " + count + "개 먹음");
}
}

6) 접근제어자

생기초

접근제어자

변수, 메서드를 누가 사용할 수 있을지 정함

접근제어자	같은 클래스	같은 패키지	자식클래스	전체
private	○	×	×	×
(생략시) default	○	○	×	×
protected	○	○	○	×
public	○	○	○	○

패키지 클래스가 저장되어 있는
파일을 모음(폴더)
패키지가 자바 문제에서
나온 경우는 없음
정적기에서는 문제에서 코드를 패키지(폴더)로 나눈 적이 없기 때문에 모두 같은 패키지에 속함
결국, 문제에서 접근제어자가 protected 혹은 생략(default)되었다면 public과 같음! → public, private 차이만 제대로 알자!
(실제로 protected, default 출제 된 적 없음)

```
class Gamja {
    public String name = "감자";
    int hp = 100;
    private String secret = "비밀";

    public void print() {
        System.out.println(name);
        showSecret();
    }

    private void showSecret() {
        System.out.println(secret);
    }
}

class AlGamja extends Gamja {
    void status() { 생략 == default == public
        showSecret(); }
    print();
}

class Main {
    public static void main(String[] args) {
        Gamja g = new Gamja();
        g.print();
        g.showSecret();
        AlGamja a = new AlGamja();
        a.status();
        System.out.println(a.hp);
    }
}
```

감자
비밀
감자
비밀
100

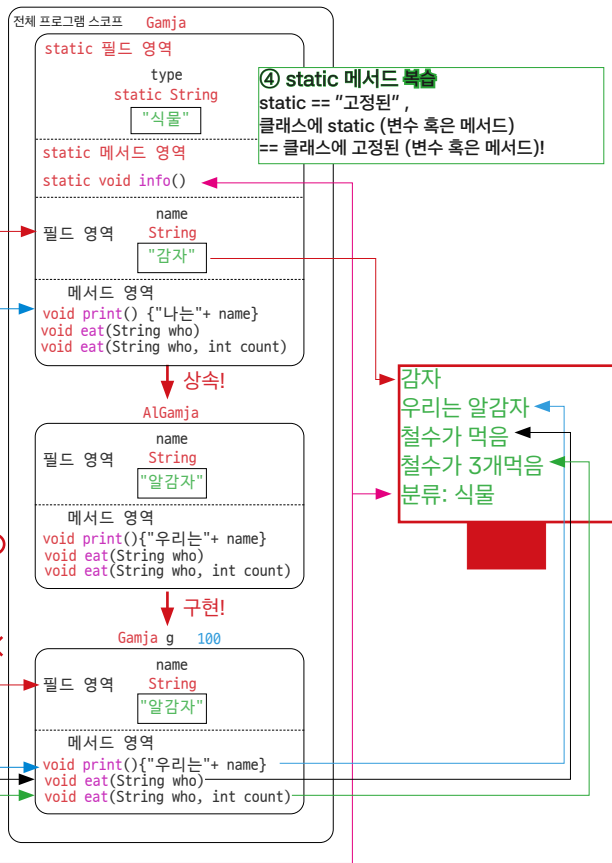
*애러코드 삭제시 출력

? 코딩 압축 PDF 어떻게 활용하나요?

핵기초 → C언어 → 자바 → 파이썬 순으로 꼭 읽어주세요! 이해되지 않는 부분이 있다면 넘어가세요! 2회독 후 읽은 이론에 관한 문제 꼭 푸세요!

? 이해가 안되는 부분이 있어요!

AI를 적극 활용하세요! 챗지피티, 클로드, 제미니이 무엇이든 상관없어요! 질문할때에는 무엇을 모르는지 정확히 알아야 해요. 특히 용어를 내가 정확히 이해하고 있는지 끊임없이 셀프 평가를 하고 모르는 용어에 대한 정의를 완벽하게 알아내면 좋아요. 특히 아는 것과 공통점, 차이점을 물어보세요!
예) 자바에서 상속이 뭐야? 클래스는 뭐야? 메서드는 뭐야? 함수는 뭐야? C언어의 함수와 자바의 메서드의 차이는 뭐야? C언어의 변수와 자바의 변수 공통점은 뭐고 차이점은 뭐야? → 그래도 이해가 안된다면 댓글로 물어봐요!



7) 참조형 vs 기본형 변수

생기초

기본형 vs 참조형

뭔가 복잡한 애들(문자열, 배열, 클래스-객체: 참조형)은 변수에 주소 복사 저장
간단한 애들(int, float, char: 기본형)은 변수에 값 복사 저장

```
class Gamja {
    public String name = "감자";
}

class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = a;
        System.out.println(b);

        Gamja g1 = new Gamja();
        Gamja g2 = g1;
        g1.name = "왕감자";
        System.out.println(g2.name);
    }
}
```

8) 배열

생기초

배열

배열 선언
int[] gamja_array = new int[3];
배열의 크기는 배열 상자 갯수(3개)
int[] == int로 이루어진 배열 클래스
배열에 담긴 int의 주소일 명세
배열도 객체! 객체의 주소
초기와 안하면 기본값 0
인덱스
*C언어와 다른점
각 변수마다 주소가 있지 않음, 대표주소(주소100) 하나만 설정
각 상자는 인덱스 접근

②
int[] gamja = new int[1,2,3];
System.out.println(gamja[0]);
System.out.println(gamja[1]);
System.out.println(gamja[2]);
1
2
3
gamja
int[] 주소
주소100

length 필드
배열 클래스에서는 length 필드가 존재
배열 상자의 갯수가 자동 계산되어 저장
int[] gamja = {10,11,12};
System.out.println(gamja.length);
3
length
int
3
gamja.length() → 애러! 메서드야!

문자열 선언 문자열 리터럴: 바로 따옴표
작성한 문자열

```
String gamja1="Gamja";
String gamja2= new String("Gamja");
```

문자열 객체

gamja1과 gamja2는 같아보이지만
다른 주소를 갖고 있음!

복습 참조형은 변수에 주소를 복사 저장!

```
System.out.println(gamja1);
System.out.println(gamja2);
```

Gamja
Gamja

```
String g1="Gamja"
String g2= new String("Gamja")
String g3="Gamja"
```

```
System.out.println(g1 == g2);
System.out.println(g1 == g3);
```

false → 주소100과 주소200은 다르다
true → 주소100과 주소100은 같다

메서드

<code>.length()</code>	문자열의 글자수	<code>g1.length()</code>	5
<code>.charAt(n)</code>	문자열의 n+1번째 글자(n은 0부터)	<code>g1.charAt(0)</code>	'G'
<code>.substring(n)</code>	n+1번 글자부터 끝까지 부분 문자열 추출	<code>g1.substring(2)</code>	"nja"
<code>.substring(n,m)</code>	n+1번부터 m+1까지 부분 문자열 추출	<code>g1.substring(2,3)</code>	"mj"
<code>.toUpperCase()</code>	String을 모두 대문자 로 변환	<code>g1.toUpperCase()</code>	"GAMJA"
<code>.toLowerCase()</code>	String을 모두 소문자 로 변환	<code>g1.toLowerCase()</code>	"gamja"
<code>.replace('c1', c2)</code>	'c1' 글자를 'c2' 글자로 변경	<code>g1.replace('a', 'z')</code>	"gzmjz"
<code>.split("c")</code>	문자열을 "c"로 쪼개기	<code>g1.split("a")</code>	{"G", "mj"}
<code>.indexOf('c')</code>	'c' 글자 위치 찾기(첫번째만 반환)	<code>g1.indexOf('a')</code>	1
<code>.contains("s")</code>	"s" 문자열 포함하나	<code>g1.contains("mj")</code>	true

```
== 은 변수에 든 값을 비교  
.equals()는 문자열의 내용 비교  
String g1="Gamja"  
String g2= new String("Gamja")
```

```
System.out.println(g1 == g2);
System.out.println(g1.equals(g2));
```

변수에 든 값이 다르네
g1==g2 하면
결과는 false

```
String g=" Gamja is crazy ";
System.out.println(g+"끝");
System.out.println(g.trim()+"끝");
```

사용자가 만들지 않아도 사용 가능한 클래스

`System.out.println()`에서 `System`도 기본제공 클래스 시험에 나왔던 기본제공 클래스 == {Integer, Math}

Integer

메서드	의미	예시	결과
<code>Integer.valueOf("문자열")</code>	문자열을 정수로 변환	<code>Integer.valueOf("1")</code>	1
<code>Integer.parseInt("문자열")</code>		<code>Integer.parseInt("2")</code>	2

Math

메서드	의미	예시	결과
Math.max(n,m)	n,m 중 큰 값	Math.max(3,5)	5
Math.min(n,m)	n,m 중 작은 값	Math.min(3,5)	3
Math.abs(n)	n의 절대값(absolute)(음수 → 양수)	Math.abs(-2)	2
		Math.abs(3)	3
Math.pow(n,m)	n의 m승(거듭제곱,power)(n^m)	Math.pow(2,3)	8
Math.round(n,m)	소수점 첫째자리에서 반올림(round)	Math.round(4.5)	5

무조건 코딩에 더 많은 비율의 공부가 필요합니다! 코딩 문제는 기초가 다져지지 않다면, 조금만 문제가 달라져도 풀지 못하게 될 수 있어요. 때문에 기초를 다지기 위해 충분히 많은 시간동안 코딩 이론을 공부하고 코드에 익숙해지고 문제푸는 과정을 익히는 시간이 필요합니다! 이론은 빠르게, 코딩은 진득하게 공부하는 것이 가장 좋은 전략이에요.

try(시도해볼게) 근데 예외나오면 catch!(잡아서) 따로 처리

try {
 System.out.println("감자");
} catch (Exception e) {
 System.out.println("예외 발생");
} finally {
 System.out.println("항상 실행");
}

예외(Exception) 객체를 매개변수에 담아서 사용 예외가 발생하면 자기가 알아서 Exception 객체를 던진다(throw)
실행하는데 예외(에러)가 없다면
try내 코드 실행
finally내 코드 실행
감자
항상 실행

예외 발생
catch내 코드 실행
예외 발생
항상 실행
finally내 코드 실행

```
try {
    System.out.println("감자");
    return 0;
} catch (Exception e) {
    System.out.println("예외 발생");
    return 0;
} finally {
    System.out.println("항상 실행");
}
```

System.out.println("감자"); 실행하는데 예외(에러)가 없다면
 System.out.println("감자"); 실행하는데 예외(에러)가 있다면
 try내 코드 실행
 예외 발생
 catch내 코드 실행
 감자 항상 실행 ← finally내 코드 실행
 예외 발생 항상 실행 ← finally내 코드 실행
 * 반환값: 0
 * 반환값: 0
 return을 만나도 finally는 return 전에 항상 실행!

```

// 왼쪽 예외를 던질 수도 있음
public static int check(int x) {
    try {
        if (x > 2) {
            throw new Exception();
        }
        return x * 2;
    } catch (Exception e) {
        return 7;
    }
}

// 오른쪽 예외를 던질 수 있음을 먼저 알려줌
// 이거 예외 생기면 다른애가 처리해- 이느낌
// 그러므로 catch가 있는 경우엔 어파피 내부에서 처리하니까
// 굳이 throws 선언 쓸 필요 없음
public static int check(int x) throws Exception {
    try {
        if (x > 2) {
            throw new Exception();
        }
        return x * 2;
    }
}

```

```
System.out.println(check(1));
System.out.println(check(3));
```

기초 조건문은 c언어 조건문 확인!(C언어와 동일)

배열 내 상자 each(각각)을 꺼냄(향상된 for문)

```
for(변수 : 배열){
반복할건데 변수는 배열에서 꺼내서

int[] gamja={10,11,12}

for(int x : gamja){
    System.out.println(x);
}
```

10
11
12

기초 반복문은 c언어 반복문 확인!(C언어와 동일)

상수를 잘 묶어서 **enumeration** 나열해놓음!

상수: 한번 정하면 변하지 않음 (변수와 반대) 혹은 변하지 않기로 해 약속

enum 초기화

상수만 나열 이넘은 보통 맨 앞 글자 대문자

enum Season {
 SPRING, SUMMER, FALL
}

상수는 보통 모두 대문자

SPRING	SUMMER	FALL
name	name	name
String	String	String
ordinal	ordinal	ordinal
int	int	int
SPRING	SUMMER	FALL
0	1	2

속성을 추가해보자 !

```
enum Season {
    SPRING("봄");
    SUMMER("여름");
    FALL("가을");
}

String korean; // 생성자 이용
// 상수에 속성 추가
Season(Season korean){
    this.korean = korean;
}
```

*클래스와 구현 관점: **SPRING, SUMMER, FALL** 상수는 사실 Season 클래스가 구현된 객체!
그러므로 Season 클래스에 구현된 속성, 생성자를 사용하여 구현된 거임

enum 메서드	의미
----------	----

<code>이념.values()</code> !	상수를 선언 순서대로 배열로 반환	<code>enum Season {SPRING, SUMMER, FALL}</code>
<code>이념.상수.name()</code>	상수의 name을 반환	<code>.values() !</code>
<code>이념.상수.ordinal()</code>	상수의 ordinal을 반환	<code>System.out.print(Season.values());</code>
<code>이념.valueOf()</code>	name으로 상수 찾아 반환	<code>[SPRING, SUMMER, FALL]</code>

The diagram illustrates the internal structure of an Enum class. It shows three Enum constants: **SPRING**, **SUMMER**, and **FALL**, each with an ordinal value (0, 1, 2 respectively). The diagram also shows how these constants are used in code, such as `Season.SPRING.ordinal()` and `Season.valueOf('SPRING')`.

Season 100

- SPRING 200**
 - name: String, ordinal: int
 - SPRING, 0
- SUMMER 300**
 - name: String, ordinal: int
 - SUMMER, 1
- FALL 400**
 - name: String, ordinal: int
 - FALL, 2

Ordinal values:

- `Season.SPRING.ordinal()` returns 0
- `Season.SUMMER.ordinal()` returns 1
- `Season.FALL.ordinal()` returns 2

ValueOf method:

- `Season.valueOf("SPRING")` returns `Season.SPRING`
- `Season.valueOf("SUMMER")` returns `Season.SUMMER`
- `Season.valueOf("FALL")` returns `Season.FALL`

14) 람다 표현식

인터페이스를 한줄로 구현할거임!(한줄요약) 사실... 한줄이 아닐수도

```

interface Edible {
    void eat(String who);
}

class Gamja implements Edible {
    @Override
    public void eat(String who) {
        System.out.println(who + "가 감자를 먹음");
    }
}
        
```

원래는 이렇게 implements 사용해서 클래스로 구현했네...

한 줄 람다 표현식 가능하면 (중괄호) 생략 가능 + return 생략 가능!

```

(n) -> { return n + 1; }
(n) -> n + 1;
        
```

필드 영역

```

Edible e = (who) -> {System.out.println(who + "가 감자를 먹음");};
        
```

메서드 영역

```

void eat(String who)
        
```

아다 쓸라나 귀찮네!

메개변수는 알아야지(타입 생략 가능) eat 메서드 내용은 알아야지

어떤 interface 구현하는지 알려주고

어짜피 거기엔 메서드 하나 있으니까 eat 메서드겠네... 그니까 메서드 이름은 생략

15) 추상 클래스

설계도(클래스)의 설계도 - 틀의 틀 틀틀

```

abstract class Food {
    String name;

    Food(String name) {
        this.name = name;
    }

    abstract void eat(String who);

    void describe() {
        System.out.println(name + "는 먹는거.");
    }
}

class Gamja extends Food {
    Gamja() {
        super("감자");
    }

    @Override
    public void eat(String who) {
        System.out.println(who + "가 감자를 먹음");
    }
}

public class Main {
    public static void main(String[] args) {
        Food f = new Gamja();

        f.eat("고양이");
        f.describe();
    }
}
        
```

추상(abstract)

추상 메서드는 반드시 구현해야함

extends 상속!

부모 클래스 상속과 다른점
→ 이 틀을 꼭 구현해야함
단, abstract한

구현!

Food f 100

고양이가 감자를 먹음
감자는 먹는거

16) 제네릭

어떤 타입(T)이던 들어올 수 있지만 사실은 그 타입이 아닐수도 있단다
제네릭은 그냥 타입을 고정하지 않고 싶어서 쓰는 거임 뿐. 범용적(Generic)인 코드를 만들고 싶어서 쓰
어떤 Type이던 여기에 넣으세요 표시

```

class Gamja<T> {
    T item;

    Gamja(T item) {
        this.item = item;
    }

    void show() {
        System.out.println("감자 안에 있는 것: " + item);
    }
}

public class Main {
    public static void main(String[] args) {
        Gamja<String> strGamja = new Gamja<>("고양이");
        Gamja<Integer> intGamja = new Gamja<>(123);

        strGamja.show(); // 감자 안에 있는 것: 고양이
        intGamja.show(); // 감자 안에 있는 것: 123
    }
}
        
```

만 있다고 당황하지 말기. 그냥 int, String 이런걸로 바꿔거임

각 타입에 맞게 생성자에 값 넣어서 호출함

이건 보살 타입 정하잖음

타입이 사라져버림?

화가나는 부분은 컴파일 끝나면 T를 모두 Object로 바꿔 버린다는 거임

T가 Object가 되어버림

컴파일

이게 오버로딩임 ㅋㅋ

오버로딩에 의해 왼쪽처럼 출력될 거 같은데 아님!

Object 고양이
Object 123

이게 정답!

이걸 근데 왜?
제네릭의 2번째 장점, 타입 안정성 때문!
Gamja<String> strGamja = new Gamja<>("고양이");
여기서 String 대신 다른 타입이 들어가면 컴파일 단계에서 에러 잡아줌!

3. 파이썬

1) 리스트

변가 == 변경가능

생기초

리스트 선언 + 초기화

```

gamja = [10, 11, 12]
print(gamja[-1])
print(gamja[0])
        
```

리스트 선언 + 초기화

가장자리도 정하지 않고 배열 크기도 정하지 않음
자유로운 배열 == 리스트

뒤에서 1번째
음수로 뒤에서 부터 접근 가능!

뒤에서 2번째

배열과 똑같이 인덱스 접근 가능

인덱스 접근

파이썬만의 특별한 접근법 존재함

gamja[0]=20
print(gamja)

인덱스로 변경 가능!

변가

대괄호까지 그대로 출력!

요소 == 성자에 담긴것

리스트 메서드

메서드 의미

.append(원든) 리스트 끝에 원든 추가

.extend(경수배고) 리스트 끝에 리스트 등 여러 요소가 포함될 수 있는 것의 요소들을 풀어서 추가

.insert(i, 원든) [i] 인덱스 위치에 원든 추가(기존에 있던거는 뒤로 밀림)

.pop(i) [i] 인덱스 위치 요소 꺼내고 제거 .pop()이면 맨 마지막 요소 제거

.remove(원든) 원든 삭제(원든이 같은게 여러 개면 맨 앞에만 삭제)

.reverse() 리스트 뒤집기

.append

```

gamja = [10, 11, 12]
gamja.append(13)
print(gamja)
        
```

추가

gamja = [10, 11, 12, 13, 14]

print(gamja)

또 다른 리스트가 통째로 추가

.extend

```

gamja = [10, 11, 12]
gamja.extend([13, 14])
print(gamja)
        
```

요소를 풀어서 추가

.insert

```

gamja = [10, 11, 12]
gamja.insert(1, [13, 14])
print(gamja)
        
```

일어내고 들어가기

기존거 인덱스 하나씩 밀림

.pop

```

gamja = [10, 11, 12]
gamja.pop()
print(gamja)
        
```

Pop!

팝은 뒤가되 맨 마지막 요소 날아감

.remove

```

gamja = [10, 11, 10]
gamja.remove(10)
print(gamja)
        
```

앞의 10만 제거, 뒤의 10은 살아남음

.reverse

```

gamja = [10, 11, 12]
gamja.reverse()
print(gamja)
        
```

뒤집기

2차원 리스트

리스트에는 원든 넣을 수 있어서 리스트 그 자체도 들어감

```

gamja_2d = [[10, 11, 12], [20, 21, 22], [30, 31, 32]]
print(gamja_2d[1])
print(gamja_2d[0][1])
        
```

행 열

요소에 접근

슬라이싱

일부분 잘라내서(slicing) 꺼내 볼 수 있음

```

gamja = [10, 11, 12, 13]
sliced_gamja = gamja[1:3]
print(sliced_gamja)
        
```

인덱스 1부터 3... 이 아니라 그 전까지!

앞에 아무것도 없으면 sliced_gamja = gamja[:3] 인덱스 0부터!

뒤에 아무것도 없으면 sliced_gamja = gamja[2:] 마지막까지!

간격 지정

풀썬은 생략기 기본 1

```

gamja = [10, 11, 12, 13, 14]
sliced_gamja = gamja[1:4:2]
print(sliced_gamja)
        
```

무조건 밟고 풀썬 풀썬

인덱스 1부터 4... 가 아니라 그 전까지! 인덱스 2풀썬 했을때 밟는 것만!

역순 슬라이싱

반대로 풀썬

```

gamja = [10, 11, 12, 13, 14]
sliced_gamja = gamja[4:1:-1]
print(sliced_gamja)
        
```

반대로 뛰어와야하니까 인덱스도 반대로! 4부터 1... 이 아니라 그 전까지!

처음은 무조건 밟고 풀썬 풀썬

처음부터 끝까지 2풀썬

Note

2) 문자열

변불==변경 불가능

생기초

문자열 선언 + 초기화

```
gamja="감자왕"
역시 자료형 정하지 않음
```

gamja

[0] [1] [2]

```
print(gamja[0])
리스트와 똑같이 인덱스 접근 가능
```

인덱스 접근

```
print(gamja[-1])
```

뒤에서 1번째

음수로 뒤에서 부터 접근 가능!

```
print(gamja[-2])
```

뒤에서 2번째

왕

자

감

```
gamja[0]="중"
print(gamja)
```

인덱스로 변경 불가능!

리스트와 차이점 why? 안전을 위한 규칙 정도도 알아두자

문자열

문자열 메서드

메서드 의미

.split(구분자)	구분자 기준으로 나눠서 리스트로 반환
.strip()	기본적으로 양끝 공백 제거(괄호에 문자 들어가면 양끝 해당 문자 제거)
.replace(old,new)	문자열의 old를 new로 바꿈
.find(찾고싶은거)	찾고 싶은거의 첫번째 인덱스를 반환. 근데 찾고 싶은게 문자열에 여러개 있으면 첫번째 거의 첫번째 인덱스(?)만 반환
.startswith(이거)	이거로 시작하는지 확인. True False 반환
.upper()	모두 대문자로
.lower()	모두 소문자로

```
gamja="감자왕"
print(gamja.split("자"))
```

gamja

[0] [1] [2]

split

[0] [1]

["감", "왕"]

```
gamja="감자왕"
print(gamja.strip())
```

gamja

[0] [1] [2] [3] [4] [5]

감자왕

감자왕

.replace

```
gamja="감자왕"
print(gamja.replace("감자", "고구마"))
```

```
gamja="감자감자왕감자"
print(gamja.find("감자"))
```

gamja

[0] [1] [2] [3] [4] [5] [6]

감자

0

찾고싶은거 중 첫번째 인덱스

```
english="gamja"
print(english.upper())
```

GAMJA

```
ENGLISH="GAMJA"
print(ENGLISH.lower())
```

gamja

```
gamja="감자감자왕감자"
print(gamja.startswith("감자"))
```

gamja

True

"감자"로 시작하네

False

"왕감자"로 시작안하잖아

가까이에 있는 문자열은 자동으로 합쳐짐

+를 쓰지 않아도 된다(자바는 + 썼음, C언어는 + 도 안됨)

```
print("감자"+"왕감자")
print("감자" "왕감자")
```

감자왕감자

감자왕감자

같음

슬라이싱

```
gamja="감자감자왕감자"
sliced_gamja=gamja[1:3]
```

gamja

[0] [1] [2] [3] [4] [5] [6]

감자

인덱스 1부터 3. 이 아니라 그 전까지!

다른 모든 슬라이싱이 리스트와 동일하니 리스트의 슬라이싱 참고 바람!

Note

3) 입력함수

생기초

입력함수

input

```
gamja=input()
print(gamja)
```

gamja

입력: 감자왕

[0] [1] [2]

감자왕

```
gamja=input("읽든 입력해:")
print(f"{gamja} is my life")
```

gamja

읽든 입력해:

입력: 감자왕

[0] [1] [2]

감자왕 is my life

4) 딕셔너리

변가

생기초

딕셔너리

키(key)와 값(value)를 갖고 있음. 영어사전보면 단어와 뜻이 붙어있는 거랑 비슷해서 딕셔너리(사전)

딕셔너리 선언 + 초기화

```
gamja_dic={"뿌리":"감자뿌리", "줄기":"감자", "꽃":"감자꽃"}
```

딕셔너리 메서드

메서드	의미
.keys()	키만 리스트처럼 반환
.values()	값만 리스트처럼 반환
.items()	키와 값을 튜플처럼 반환

키를 리스트, 딕셔너리, 집합 안됨	읽든 넣어도 됨!
gamja_dic	gamja_dic
변불	변가
변불	변가
키를 인덱싱하듯이 사용할 거	

.keys	.values
print(gamja_dic.keys())	print(gamja_dic.values())
["뿌리", "줄기", "꽃"]	["감자뿌리", "감자", "감자꽃"]

.items
print(gamja_dic.items())
[("뿌리", "감자뿌리"), ("줄기", "감자"), ("꽃", "감자꽃")]

5) 집합

변가

생기초

집합

딕셔너리처럼 {} 사용해서 선언, 내부에 있는 거(요소)에 대해 중복허용 안함 + 순서 없음

집합 선언 + 초기화

```
gamja_set={"감자뿌리", "감자", "감자꽃"}
```

gamja_set

```
gamja_set=set(["감자뿌리", "감자", "감자꽃"])
```

내장함수 set을 활용해서 선언 가능(리스트 활용)

["감자뿌리", "감자꽃"]

["감자"]

집합 메서드

메서드 의미

.add(요소)	요소 추가
.update(여러 요소가 담긴거)	여러 요소 풀어서 한 방에 추가
.remove(요소)	요소 제거(제거할 요소가 없으면 오류)
.discard(요소)	요소 제거(제거할 요소가 없어도 오류 없음)

```
gamja_set={"감자뿌리", "감자", "감자꽃"}
gamja_set.add(1)
gamja_set.update([1,2])
```

```
gamja_set.remove("감자꽃")
gamja_set.discard("감자꽃")
gamja_set.remove("왕감자")
```

...조용... 아무 일도 없었다. (사실 None이라는 '아무것도 없음'을 반환하긴함)

집합 연산

비트연산자인데 양옆에 집합이 있으면 집합 연산자로 활용

```
set1={1,2,3}
set2={3,4,5}
```

print(set1 & set2)

print(set1 | set2)

print(set1 - set2)

print(set1 ^ set2)

? 프로그래밍(코딩 - c언어, 자바, 파이썬)은 어떻게 공부하는게 좋을까요?

기술 문제에서 프로그래밍 부분만 풀어보세요. 특히 최신 기술 문제부터 푸는 걸 추천드립니다. 어려운 문제는 일단 풀지 말고 각 시험당 쉬운 문제 3개만 맞추세요!
(절대 C언어, 자바, 파이썬 중 하나만 파서 그 부분을 많이 맞춰야겠다고 생각하면 안되요!)

? C언어, 자바, 파이썬 중 2개만 집중적으로 공부하는게 더 효율적일까요?

아니요! 물론 2개 언어를 완벽하게 안다면 합격하겠지만, 2개 언어를 완벽하게 하는 것보다 3개 언어의 기초를 탄탄히 하는게 훨씬 쉽고 오히려 더 많은 문제를 맞출 확률이 높습니다! 더군다나 세언어는 비슷한점이 많다는 점!

6) 튜플

() 사용해서 선언, 내부에 있는 거(요소)변경 못함

튜플 선언 + 초기화

```
gamja_tuple=(10,11,12)
gamja_tuple
```

```
(10 11 12)
[0] [1] [2]
```

인덱스 접근

```
print(gamja_tuple[1])
```

```
11
```

리스트, 문자열과 같이 요소 접근도 가능!

gamja_tuple[0]=20

인덱스로 변경 불가능!

에러!

슬라이싱

리스트, 문자열과 동일

```
gamja_tuple=(10,11,12,13)
print(gamja_tuple[1:3])
```

```
(11,12)
```

```
gamja_tuple
(10 11 12 13)
[0] [1] [2] [3]
```

다른 모든 슬라이싱이
리스트와 동일하니
리스트의 슬라이싱 참고 바람!

괄호까지
그대로 출력!

인덱스 1부터 3... 이 아니라 그 전까지!

튜플 연산

더하기(+)

두 튜플 이어붙여서 새로운 튜플

```
num = (10,11)
gamja = ("감자", "왕감자")
sum = num + gamja
print(sum)
```

```
sum
(10 11 "감자" "왕감자")
[0] [1] [2] [3]
```

(10,11,"감자","왕감자")

곱하기(*)

튜플 요소를 반복해서 붙이기

```
num = (10,11)
multiple = num * 3
print(multiple)
```

```
multiple
(10 11 10 11 10 11)
[0] [1] [2] [3] [4] [5]
```

(10,11,10,11,10,11)

튜플 메서드

메서드 의미

.count(요소) 요소가 몇개?

.index(요소) 요소 위치(인덱스)는 어디?

.count

```
gamja = (10,11,"감자","감자","왕감자")
print(gamja.count("감자"))
```

```
2
```

.index

```
print(gamja.index("왕감자"))
```

```
4
```

```
gamja
(10 11 "감자" "감자" "왕감자")
[0] [1] [2] [3] [4]
```

7) 기본 제공 함수

함수	예시	의미	반환
range(3)		0부터 3 전까지 정수 반환	0,1,2
range()	range(1,6)	1 부터 6 전까지 정수 반환	1,2,3,4,5
	range(1,6,2)	1 부터 6 전까지 2 간격으로 정수 반환	1,3,5
type()	type(1)==type("감자")	자료형 반환(int,float,list,str,...)	False
len()	len((10,20,"감자"))	몇몇 길이(요소 개수)가 얼마인지 반환	3
sum()	sum({10,20,30})	몇몇 내부에 정수만 있으면 다 더해줌	60
all()	all([True,True])	요소가 전부 참이면 참 반환	True
any()	any([False,False,True])	요소중 하나라도 참이면 참 반환	True
map(함수, 여러 요소가 포함될 수 있는 것)		함수에 요소 하나 하나씩 사용(람다 표현식 참고)	
enumerate(여러 요소가 포함될 수 있는 것)		(인덱스, 값) 반환(언패킹 참고)	

8) 조건문

if ! 조건문 참이면 if 내부 코드 실행

```
gamja1 = 1
gamja2 = 1
if문(함수)의 시작 알림
```

파이썬은 들여쓰기가 중요!
들어쓰기가 끝나면 if문(함수) 끝
다음 if문(함수) 시작

if gamja1 > 0: (참)
→ print("큰 감자")
if gamja2 > 1: (거짓)
→ print("1보다 큰 감자")

if gamja > 2: (이거? 아니네...)
→ print("if 참")
elif gamja > 1: (그럼 이거? elif(else if)
→ print("elif 참")
else: (else 밖에는...)
→ print("else 밖에는...")

if-else 조건문 거짓이면 else 내부 코드 실행

```
gamja = -1
if gamja > 0: (거짓)
    print("큰 감자")
else: (실행)
    print("작은 감자")
```

같은 작은 감자

```
gamja = -1
if gamja > 0: (if가 참일때 실행) if 조건 else (if 거짓일때 실행)
    print("큰 감자") if gamja > 0 else print("작은 감자")
    print("큰 감자")
```

작은 감자

print("큰 감자") 할거야. gamja > 0 일때만. 아니면 print("작은 감자")

9) 반복문

반복문	설명	예시
for - in	for 변수 in 반복할 대상	for i in range(3):
while	조건식만 존재(C언어, 자바와 동일)	while i<2:
break	루프 즉시 종료	while i<2: if(i==1): break
continue	다음 반복으로 건너뛰기	while i<2: if(i==1): continue

? 시간이 너무 부족해요 하지만 포기할래요

정말 정말 시간이 부족하다면 차라리 파이썬을 포기하세요!
최근 파이썬 기출문제 범위가 크게 늘어난 경향이 있습니다!
가장 문제가 적은 수(1~2문제)로 나오는 파이썬을 포기하는게...

10) 컴프리헨션

한 줄로 여러 요소가 담기는 거를 만드는 방법(with 반복문)

리스트 컴프리헨션

```
gamja = [1,2,3]
result = [n * 2 for n in gamja]
```

리스트니까 []

[n * 2를 할거야. n 전부 다. gamja에 있는-]

if 추가

```
gamja = [1,2,3]
result = [n * 2 for n in gamja if n < 3]
```

해석해보자

[n * 2를 할거야. n 전부 다. gamja에 있는. n < 3 일때만]

딕셔너리 컴프리헨션

딕셔너리 컴프리헨션

```
gamja = [1,2,3]
result = {n:n * 2 for n in gamja}
```

딕셔너리니까 {}

{n이 키(key): 값은 n * 2를 할거야. n 전부 다. gamja에 있는.}

if 추가

```
gamja = [1,2,3]
result = {n:n * 2 for n in gamja if n < 3}
```

해석해보자

{n이 키(key): 값은 n * 2를 할거야. n 전부 다. gamja에 있는. n < 3 일때만}

제네레이터 표현식

컴프리헨션은 결과를 어딘가(리스트, 딕셔너리 등)에 담아놓지만 제네레이터는 그냥 만들고 쓰고 끝!

그래서 보통 sum(), any(), all() 등 기본 함수와 많이 쓰임

```
gamja = [1,2,3]
print(any(n > 2 for n in gamja))
gamja = [1,2,3]
print(all(n > 2 for n in gamja))
```

any

```
gamja
[1 2 3]
[0] [1] [2]
n > 2 n > 2 n > 2
X X O
[False False True]
```

만들어져서 바로 (any에) 쓰고 끝

True인게 있네! True

all

```
gamja
[1 2 3]
[0] [1] [2]
n > 2 n > 2 n > 2
X X O
[False False True]
```

전부다 True가 아니네! False

sum

```
gamja = [1,2,3]
print(sum(n for n in gamja))
```

```
gamja
[1 2 3]
[0] [1] [2]
1 2 3
6
```

if 추가

```
gamja = [1,2,3,4,5]
print(sum(n for n in gamja if n % 2 == 1))
```

```
gamja
[1 2 3 4 5]
[0] [1] [2] [3] [4]
n % 2 n % 2 n % 2 n % 2 n % 2
== 1 == 0 == 1 == 0 == 1
O X O X O
[1 3 5]
9
```

11) 언패킹

묶어있던걸 풀어서(unpack) 한방에 담는 것

```
gamja, koguma = 10, 20
print(gamja)
print(koguma)
10 20
10,20은 사실 튜플임. (10,20)과 같음
```

```
gamja, koguma, oksusu = [10, 20, 30]
print(gamja)
print(koguma)
print(oksusu)
10 20 30
```

리스트도 가능! 여러 요소가 들어가는건 다 가능

enumerate

알려한다. 숫자를 일일이 꺼낸다는 뜻. (인덱스, 값) 반환

```
gamja_num = [10, 20, 30]
for index, num in enumerate(gamja_num):
    print(index, num)
```

```
index num
0 10
1 20
2 30
```

리스트가 갖고있는 인덱스를 반환하자

튜플도 갖고있는 인덱스를 반환할거

for문 1회차

for문 2회차

for문 3회차

12) 함수

함수 정의

```
def gamja(a, b):
    return a + b
```

정의(define)

def gamja(a, b):

매개변수 함수의 시작 알림

return a + b

전체 프로그램 스코프

gamja 함수 스코프

result

8

a b

3 5

8

람다 표현식

```
gamja = lambda (a, b): a + b
result = gamja(3, 5)
print(result)
```

함수 이름 제거 + 매개변수와 return만 존재

함수 이름은 람다식 밖에... 안쓰는 경우는 아래

8

map과 사용하는 람다

```
gamja=[1,2,3,4]
result = list(map(lambda g : g + 100, gamja))
print(result)
```

요소 하나 하나씩 사용

매개변수!

함수

map 결과는 꼭 리스트, 튜플 등으로 변환해야 보임!

map! result

101 102 103 104

1 2 3 4

0 1 2 3

13) 클래스 기초

자바와 비슷함. 객체를 만들기 위한 설계도!
설계도==클래스(Class)
설계도로 실제 만든거==객체(Object)

파이썬

```
class Modoom:
    def __init__(self, g, alg, ssid):
        self.g = g
        self.alg = alg
        self.ssid = ssid

def can_gamja_print(self):
    print(f"{self.g + self.alg + \
self.ssid}개 됐다!")
```

자바

```
class Modoom{
    int g;
    int alg;
    int ssid;

    생성자
    Modoom(int g, int alg, int ssid){
        this.g=g;
        this.alg=alg;
        this.ssid=ssid;
    }

    void can_gamja_print(){
        System.out.print(
            gamja + algamja + ssidgamja + "개 됐다!"
        );
    }
}
```

파이썬화

백슬래시로 다음줄과 연결
아래 문장을 두줄로 쓰기 위해

파이썬은 줄바꿈, 들여쓰기로 문장 끝을 판단
줄바꿈하지만 이어지는 문장임을 알려주기 위한



목차를 보며 복습해봅시다([3]= 3가지)

0. 해기초
 - 1) 변수
 - 변수 선언 **C언어 자바**
 - 변수 초기화 **C언어 자바**
 - 변수 선언 + 초기화 **파이썬**
 - 2) 자료형 [7] **C언어 자바**
 - 3) 출력함수 [6] **C언어 자바 파이썬**
 - 서식지정자 [6] **C언어 자바**
 - printf + 서식지정자 **C언어 자바**
 - printf는 자체 줄바꿈이 없다 **C언어 자바**
 - print vs println **자바**
 - '+' 출력 결합 **자바**
 - putchar **C언어**
 - puts **C언어**
 - print **파이썬**
 - f-string과 %포매팅 **파이썬**
 - 4) 연산자 **C언어 자바 파이썬**
 - 산술 연산자 [5]
 - 대입 연산자 [1]
 - 복합대입 연산자 [7]
 - 비교 연산자 [3]
 - 논리 연산자 [3]
 - 비트 연산자 [4] **자바**
 - true, false엔 비트연산자 사용가능
 - 시프트 연산자 [2]
 - 연산자 우선순위 [7]
1. C언어
 - 1) 함수
 - 함수 + 변수 + 출력
 - 변수 정의 및 초기화
 - 매개변수 대입
 - 함수 종류 [3]
 - 2) 변수 스코프
 - 3) 조건문 [3] **C언어 자바**
 - if
 - if-else if
 - 삼항연산자
 - switch-case
 - 떨어져서 통과해버리기
 - default
 - 4) 반복문 [4] **C언어 자바**
 - 증감연산자
 - 대입연산자와 함께라면
 - 단독 사용
 - 5) 배열
 - 배열 선언
 - 배열 초기화
 - 6) 포인터
 - 포인터 선언
 - 포인터 초기화
 - 포인터로 값 가져오기
 - 배열을 가리키는 포인터
 - 포인터 산술
 - 7) 구조체
 - 구조체 선언
 - 구조체 초기화
 - 구조체를 포인터로 사용해봄
 - 배열이 있는 구조체
 - 8) typedef
2. 자바
 - 1) 클래스와 객체
 - 클래스
 - 객체
 - 생성자
 - 생성자를 넣어서 객체를 다르게 초기화
 - 생성자를 연결연결 해보자
 - 2) static
 - 3) 상속
 - super
 - 4) 인터페이스
 - 5) 다형성과 부모 자식 대결
 - 부모인데 객체는 자식
 - 오버라이딩
 - 오버로딩
 - 6) 접근제어자 [4]
 - 7) 참조형 vs 기본형 변수
 - 8) 배열
 - 배열 선언
 - 배열 초기화
 - length 필드
 - 9) 문자열
 - 문자열 선언
 - 문자열 리터럴은 String Pool에서 공유
 - 문자열 메서드 [10]
 - 10) 기본제공 클래스

- Integer
 - Math
- 11) 예외처리
 - finally나 return이나
 - 예외 던져!(실행) throw!
 - 이걸 예외를 던질거야(선언) throws
 - 12) for-each
 - 13) enum 이넘
 - enum 초기화
 - 속성을 추가해보자
 - enum 메서드 [4]
 - 14) 람다 표현식
 - 15) 추상 클래스
 - 16) 제네릭

3. 파이썬

- 1) 리스트
 - 리스트 선언 + 초기화
 - 인덱스 접근
 - 리스트 메서드 [6]
 - .append
 - .extend
 - .insert
 - .pop
 - .remove
 - .reverse
 - 2차원 리스트
 - 슬라이싱
 - 간격 지정
 - 역순 슬라이싱
- 2) 문자열
 - 문자열 선언 + 초기화
 - 인덱스 접근
 - 문자열 메서드 [7]
 - .split
 - .strip
 - .replace
 - .find
 - .startswith
 - .upper
 - .lower
 - 가까이에 있는 문자열 자동으로 합쳐짐
 - 슬라이싱
- 3) 입력함수 [1]
- 4) 딕셔너리
 - 딕셔너리 메서드 [3]
 - .keys
 - .values
 - .items
- 5) 집합
 - 집합 메서드 [4]
 - .add
 - .update
 - .remove
 - .discard
 - 집합 연산 [4]
- 6) 튜플
 - 튜플 선언 + 초기화
 - 인덱스 접근
 - 슬라이싱
 - 튜플 연산
 - 튜플 메서드 [2]
 - .count
 - .index
- 7) 기본 제공 함수 [8]
- 8) 조건문 [3]
 - if
 - if-else
 - if-elif
- 9) 반복문 [4]
- 10) 컴프리헨션
 - 리스트 컴프리헨션
 - 딕셔너리 컴프리헨션
 - 제네레이터 표현식
- 11) 언패킹
 - enumerate
- 12) 함수
 - 함수 정의
 - 람다 표현식
 - map과 사용하는 람다
- 13) 클래스
 - 클래스를 객체로 구현

합격을 응원합니다!